

Dispense LLM — edizione docente

Practitioner e AI Engineer · teoria, matematica, laboratori e Pollicino

TheBitPoets

Edizione 2026/27 — content pack 0.9.0

Indice

1	Come usare queste dispense	7
2	Moduli del corso	8
3	M00 — Orientamento e baseline	9
3.1	Obiettivi osservabili	9
3.2	Problema iniziale	9
3.3	Teoria Practitioner	9
3.4	Esempio minimo	9
3.5	Esempio realistico	10
3.6	Livello AI Engineer: validità dell'esperimento	10
3.7	Errori frequenti	10
3.8	Esercizi A-F	10
3.9	Laboratorio	10
3.10	Verifica rapida	11
3.11	Sintesi inclusiva	11
3.12	Fonti e collegamenti	11
4	M01 — Mappa dell'ecosistema	12
4.1	Obiettivi osservabili	12
4.2	Problema iniziale	12
4.3	Teoria Practitioner	12
4.4	Esempio minimo	13
4.5	Esempio realistico	13
4.6	Livello AI Engineer: confini e deployment	13
4.7	Confronto tra soluzioni	13
4.8	Errori frequenti	14
4.9	Esercizi A-F	14
4.10	Laboratorio	14
4.11	Verifica rapida	14
4.12	Sintesi inclusiva	14
4.13	Fonti e collegamenti	14
5	M02 — Predire il simbolo successivo	15
5.1	Obiettivi osservabili	15
5.2	Problema iniziale	15
5.3	Teoria Practitioner	15
5.4	Esempio minimo	16
5.5	Esempio realistico	16
5.6	Livello AI Engineer: matematica	16
5.7	Dalle probabilità ai bit	17
5.8	Errori frequenti	17
5.9	Esercizi A-F	17
5.10	Laboratorio	17
5.11	Verifica rapida	17
5.12	Sintesi inclusiva	17
5.13	Fonti e collegamenti	17

6	M03 — Token, byte ed embedding	18
6.1	Obiettivi osservabili	18
6.2	Problema iniziale	18
6.3	Teoria Practitioner	18
6.4	Esempio minimo	19
6.5	Esempio realistico	19
6.6	Livello AI Engineer: tokenizzazione ed embedding	19
6.7	Confronto tra rappresentazioni	19
6.8	Errori frequenti	20
6.9	Esercizi A-F	20
6.10	Laboratorio	20
6.11	Verifica rapida	20
6.12	Sintesi inclusiva	20
6.13	Fonti e collegamenti	20
7	M04 — Apprendere dai dati	21
7.1	Obiettivi osservabili	21
7.2	Problema iniziale	21
7.3	Teoria Practitioner	21
7.4	Esempio minimo	21
7.5	Esempio realistico	22
7.6	Livello AI Engineer: gradienti e ottimizzazione	22
7.7	Come leggere le curve	22
7.8	Errori frequenti	22
7.9	Esercizi A-F	22
7.10	Laboratorio	22
7.11	Verifica rapida	23
7.12	Sintesi inclusiva	23
7.13	Fonti e collegamenti	23
8	M05 — Attention e Transformer	24
8.1	Obiettivi osservabili	24
8.2	Problema iniziale	24
8.3	Teoria Practitioner	24
8.4	Esempio minimo	25
8.5	Esempio realistico	25
8.6	Livello AI Engineer: equazioni e shape	25
8.7	Confronto tra implementazioni	26
8.8	Errori frequenti	26
8.9	Esercizi A-F	26
8.10	Laboratorio	26
8.11	Verifica rapida	26
8.12	Sintesi inclusiva	26
8.13	Fonti e collegamenti	26
9	M06 — Architetture moderne	27
9.1	Obiettivi osservabili	27
9.2	Problema iniziale	27
9.3	Teoria Practitioner	27
9.4	Esempio minimo	27
9.5	Esempio realistico	28
9.6	Livello AI Engineer: costi e compatibilità	28
9.7	Errori frequenti	28
9.8	Esercizi A-F	28
9.9	Laboratorio	28
9.10	Verifica rapida	28
9.11	Sintesi inclusiva	28
9.12	Fonti e collegamenti	29
10	M07 — Pre-training, dati e scaling	30
10.1	Obiettivi osservabili	30

10.2	Problema iniziale	30
10.3	Teoria Practitioner	30
10.4	Esempio minimo	30
10.5	Esempio realistico	30
10.6	Livello AI Engineer: budget e contaminazione	31
10.7	Errori frequenti	31
10.8	Esercizi A-F	31
10.9	Laboratorio	31
10.10	Verifica rapida	31
10.11	Sintesi inclusiva	31
10.12	Sonti e collegamenti	31
11	M08 — Post-training e reasoning	33
11.1	Obiettivi osservabili	33
11.2	Problema iniziale	33
11.3	Teoria Practitioner	33
11.4	Esempio minimo	33
11.5	Esempio realistico	33
11.6	Livello AI Engineer: obiettivi	34
11.7	Errori frequenti	34
11.8	Esercizi A-F	34
11.9	Laboratorio	34
11.10	Verifica rapida	34
11.11	Sintesi inclusiva	34
11.12	Sonti e collegamenti	35
12	M09 — Pesì, formati e licenze	36
12.1	Obiettivi osservabili	36
12.2	Problema iniziale	36
12.3	Teoria Practitioner	36
12.4	Esempio minimo	36
12.5	Esempio realistico	37
12.6	Livello AI Engineer: ispezione e conversione	37
12.7	Scheda di decisione	37
12.8	Errori frequenti	37
12.9	Esercizi A-F	37
12.10	Laboratorio	37
12.11	Verifica rapida	38
12.12	Sintesi inclusiva	38
12.13	Sonti e collegamenti	38
13	M10 — Hardware e quantizzazione	39
13.1	Obiettivi osservabili	39
13.2	Problema iniziale	39
13.3	Teoria Practitioner	39
13.4	Esempio minimo	39
13.5	Esempio realistico	39
13.6	Livello AI Engineer: stime e roofline	40
13.7	Protocollo di misura	40
13.8	Errori frequenti	40
13.9	Esercizi A-F	40
13.10	Laboratorio	40
13.11	Verifica rapida	40
13.12	Sintesi inclusiva	41
13.13	Sonti e collegamenti	41
14	M11 — Ollama e inferenza locale	42
14.1	Obiettivi osservabili	42
14.2	Problema iniziale	42
14.3	Teoria Practitioner	42
14.4	Esempio minimo	42

14.5	Esempio realistico	43
14.6	Livello AI Engineer: template e lifecycle	43
14.7	Gestione sicura degli errori	43
14.8	Errori frequenti	43
14.9	Esercizi A-F	43
14.10	Laboratorio	43
14.11	Verifica rapida	44
14.12	Sintesi inclusiva	44
14.13	Sonti e collegamenti	44
15	M12 — Sampling e prompting	45
15.1	Obiettivi osservabili	45
15.2	Problema iniziale	45
15.3	Teoria Practitioner	45
15.4	Esempio minimo	45
15.5	Esempio realistico	45
15.6	Livello AI Engineer: decoder e vincoli	46
15.7	Errori frequenti	46
15.8	Esercizi A-F	46
15.9	Laboratorio	46
15.10	Verifica rapida	46
15.11	Sintesi inclusiva	46
15.12	Sonti e collegamenti	46
16	M13 — Applicazioni conversazionali	47
16.1	Obiettivi osservabili	47
16.2	Problema iniziale	47
16.3	Teoria Practitioner	47
16.4	Esempio minimo	47
16.5	Esempio realistico	48
16.6	Livello AI Engineer: architettura	48
16.7	Struttura consigliata	48
16.8	Errori frequenti	48
16.9	Esercizi A-F	48
16.10	Laboratorio	48
16.11	Verifica rapida	49
16.12	Sintesi inclusiva	49
16.13	Sonti e collegamenti	49
17	M14 — Valutazione	50
17.1	Obiettivi osservabili	50
17.2	Problema iniziale	50
17.3	Teoria Practitioner	50
17.4	Esempio minimo	50
17.5	Esempio realistico	50
17.6	Livello AI Engineer: statistica e giudici	51
17.7	Matrice di decisione	51
17.8	Errori frequenti	51
17.9	Esercizi A-F	51
17.10	Laboratorio	51
17.11	Verifica rapida	51
17.12	Sintesi inclusiva	52
17.13	Sonti e collegamenti	52
18	M15 — Embedding, ricerca e RAG	53
18.1	Obiettivi osservabili	53
18.2	Problema iniziale	53
18.3	Teoria Practitioner	53
18.4	Esempio minimo	54
18.5	Esempio realistico	54
18.6	Livello AI Engineer: retrieval e metriche	54

18.7	Sicurezza RAG	54
18.8	Errori frequenti	55
18.9	Esercizi A-F	55
18.10	Laboratorio	55
18.11	Verifica rapida	55
18.12	Sintesi inclusiva	55
18.13	Sonti e collegamenti	55
19	M16 — Tool use, agenti e MCP	56
19.1	Obiettivi osservabili	56
19.2	Problema iniziale	56
19.3	Teoria Practitioner	56
19.4	Esempio minimo	56
19.5	Esempio realistico	56
19.6	Livello AI Engineer: macchina a stati	57
19.7	Threat model essenziale	57
19.8	Errori frequenti	57
19.9	Esercizi A-F	57
19.10	Laboratorio	57
19.11	Verifica rapida	57
19.12	Sintesi inclusiva	57
19.13	Sonti e collegamenti	58
20	M17 — Fine-tuning e adapter	59
20.1	Obiettivi osservabili	59
20.2	Problema iniziale	59
20.3	Teoria Practitioner	59
20.4	Esempio minimo	59
20.5	Esempio realistico	59
20.6	Livello AI Engineer: LoRA e training	60
20.7	Albero decisionale	60
20.8	Errori frequenti	60
20.9	Esercizi A-F	60
20.10	Laboratorio	60
20.11	Verifica rapida	60
20.12	Sintesi inclusiva	61
20.13	Sonti e collegamenti	61
21	M18 — Sistemi e kernel d'inferenza	62
21.1	Obiettivi osservabili	62
21.2	Problema iniziale	62
21.3	Teoria Practitioner	62
21.4	Esempio minimo	63
21.5	Esempio realistico	63
21.6	Livello AI Engineer: arithmetic intensity	63
21.7	Dal reference al kernel	63
21.8	Errori frequenti	64
21.9	Esercizi A-F	64
21.10	Laboratorio	64
21.11	Verifica rapida	64
21.12	Sintesi inclusiva	64
21.13	Sonti e collegamenti	64
22	M19 — Costruire e integrare: capstone Pollicino	65
22.1	Obiettivi osservabili	65
22.2	Problema iniziale	65
22.3	Capstone Practitioner: applicazione locale	65
22.4	Capstone AI Engineer: piccolo LLM da zero	66
22.5	Ramo Pollicino: dalla previsione ai bit	66
22.6	Determinismo necessario	66
22.7	Esempio minimo	67

22.8	Esempio realistico	67
22.9	Piano del futuro codec neurale	67
22.10	Errori frequenti	67
22.11	Esercizi A-F	67
22.12	Laboratorio e verifica	67
22.13	Sintesi inclusiva	68
22.14	Sontori e collegamenti	68
23	Glossario essenziale LLM	69
24	Apparati riservati al docente	72
25	Guida docente	73
25.1	Obiettivo e tono	73
25.2	Struttura di una lezione da due ore	73
25.3	Sequenza dell'anno	73
25.4	Differenziazione a due livelli	73
25.5	Minori, privacy e uso responsabile	73
25.6	Profili hardware	74
25.7	Valutazione	74
25.8	Provenienza delle immagini	74
26	Guida docente ai laboratori	75
26.1	Preparazione comune	75
26.2	Failure injection e criteri	75
26.3	Evidenze minime	75
26.4	Correzione	75
27	Soluzioni delle verifiche rapide	76
27.1	M00	76
27.2	M01	76
27.3	M02	76
27.4	M03	76
27.5	M04	76
27.6	M05	77
27.7	M06	77
27.8	M07	77
27.9	M08	77
27.10	M09	77
27.11	M10	77
27.12	M11	77
27.13	M12	77
27.14	M13	78
27.15	M14	78
27.16	M15	78
27.17	M16	78
27.18	M17	78
27.19	M18	78
27.20	M19	78
28	Diagnostica iniziale e finale	79
28.1	Chiave e punteggio	79
29	Prova pratica finale Practitioner	80
29.1	Consegna	80
29.2	Rubrica /100	80
29.3	Variante AI Engineer	80

Capitolo 1

Come usare queste dispense

Ogni capitolo offre un percorso **Practitioner**, intuitivo e pratico, e un approfondimento **AI Engineer** con matematica e implementazione. Gli esercizi A-F seguono la tassonomia TheBitLab: osserva, modifica, crea, diagnostica, mini-progetto e prodotto integrato.

I nomi e le capacità dei modelli cambiano rapidamente: consultare il catalogo datato nel repository e verificare sempre documentazione e licenze correnti. I risultati hardware non presenti in un manifest di evidenza sono stime, non misure.

Capitolo 2

Moduli del corso

Ogni modulo usa la stessa struttura a doppio livello. **Practitioner** indica ciò che deve saper fare uno studente; **AI Engineer** aggiunge matematica, implementazione e ricerca. Le verifiche richiedono evidenza osservabile.

Modulo	Titolo	Ore P	Ore AE
M00	Orientamento e baseline	2	4
M01	Mappa dell'ecosistema	2	4
M02	Predire il simbolo successivo	3	8
M03	Token, byte ed embedding	3	10
M04	Apprendere dai dati	4	12
M05	Attention e Transformer	5	16
M06	Architetture moderne	4	12
M07	Pre-training, dati e scaling	3	10
M08	Post-training e reasoning	3	12
M09	Pesi, formati e licenze	3	8
M10	Hardware e quantizzazione	4	12
M11	Ollama e inferenza locale	4	8
M12	Sampling e prompting	3	8
M13	Applicazioni conversazionali	4	10
M14	Valutazione	4	14
M15	Embedding, ricerca e RAG	4	14
M16	Tool use, agenti e MCP	3	12
M17	Fine-tuning e adapter	3	14
M18	Sistemi e kernel d'inferenza	3	18
M19	Costruire e integrare	3 + progetto	24 + progetto

Le ore AI Engineer includono le ore Practitioner quando il concetto è comune.

Capitolo 3

M00 — Orientamento e baseline

Domanda guida: come studiamo un sistema che produce risposte convincenti senza confondere fluidità e conoscenza? **Durata:** 2 ore Practitioner; 4 ore AI Engineer. **Prerequisiti:** curiosità, uso elementare del computer e disponibilità a verificare le affermazioni.

3.1 Obiettivi osservabili

Al termine saprai distinguere modello, applicazione e servizio; formulare una previsione verificabile; registrare una baseline; classificare un risultato come misurato, simulato o atteso; applicare regole minime su privacy, copyright e sicurezza. Nel livello AI Engineer saprai inoltre descrivere minacce, variabili di confondimento e limiti di validità di un esperimento.

3.2 Problema iniziale

Due chatbot rispondono correttamente alla stessa domanda. Possiamo concludere che sono equivalenti? No: potrebbero usare modelli diversi, retrieval, strumenti esterni o istruzioni nascoste. Anche a parità di risposta potrebbero differire per costo, latenza, privacy, memoria, licenza e affidabilità su casi nuovi. Il corso parte quindi da una regola: **una demo non è ancora un'evidenza generale**.

3.3 Teoria Practitioner

Un modello linguistico è una funzione parametrica che, dato un contesto, assegna probabilità ai possibili token successivi. L'applicazione decide come raccogliere il prompt, conservare la conversazione, recuperare documenti e mostrare l'output. Il runtime carica ed esegue i pesi. Il servizio aggiunge rete, autenticazione, quote e condizioni economiche. Dire “uso l'AI” nasconde questi livelli e rende impossibile capire che cosa è accaduto.

Una baseline è il punto di confronto più semplice e onesto. Per riassumere un testo può essere “prime tre frasi”; per classificare email può essere la classe più frequente; per Pollicino può essere il file non compresso o un compressore tradizionale. Senza baseline, “funziona bene” non ha significato operativo.

Ogni prova usa il ciclo **domanda → ipotesi → procedura → osservazione → conclusione limitata**. Una misura è prodotta dall'esecuzione reale; una simulazione deriva da un modello esplicito; un'aspettativa è ciò che prevediamo prima della prova. Mescolarle è uno degli errori più comuni nei progetti AI.

3.4 Esempio minimo

Domanda: “Abbassare la temperatura rende identica una risposta locale?” Ipotesi: “con temperatura zero l'output sarà sempre identico”. Procedura: stessa revisione del modello, stesso

prompt, stesso template, stessi parametri, cinque esecuzioni. Osservazione: si registrano hash e testi. Conclusione corretta: “nel nostro runtime e hardware, con questa configurazione, le cinque uscite coincidono” oppure “non coincidono”. Non possiamo trasformarla in “tutti gli LLM sono deterministici”.

3.5 Esempio realistico

Devi scegliere un assistente locale per documenti scolastici. Prima stabilisci criteri: nessun dato personale verso cloud, risposta entro 10 secondi, citazioni recuperabili, memoria entro il budget. Poi confronti una baseline senza modello, un modello piccolo e uno più grande sullo stesso set di richieste. Conservi versione, parametri, hardware, dataset e risultati. La scelta finale nasce dai vincoli, non dal modello più famoso.

3.6 Livello AI Engineer: validità dell'esperimento

La variabile indipendente è ciò che modifichi, per esempio la quantizzazione; le variabili dipendenti sono misure come latenza e accuratezza. Tutto il resto dovrebbe rimanere controllato. Se cambi insieme modello, prompt e runtime non puoi attribuire il risultato a una causa.

Definisci prima metrica e soglia. Per una proporzione di successi $\hat{p} = k/n$, un campione piccolo ha grande incertezza: 9 risposte corrette su 10 non dimostrano un'affidabilità del 90% sul mondo reale. Dataset, campionamento e failure case contano quanto il punteggio medio. Registra sempre semi casuali quando disponibili, commit, dipendenze e condizioni hardware.

Una threat model minima considera dati sensibili nel prompt, output falso o dannoso, dipendenze compromesse, licenze incompatibili, prompt injection e accesso eccessivo a strumenti. Il controllo deve essere proporzionato al rischio: una spiegazione didattica e un agente che può modificare file non possono avere la stessa autonomia.

3.7 Errori frequenti

- Chiamare “modello” l'intera applicazione.
- Scegliere la metrica dopo aver visto il risultato.
- Confrontare prove con prompt, contesto o hardware diversi.
- Conservare soltanto lo screenshot migliore.
- Inserire dati personali, password o documenti riservati nei prompt.
- Trattare sicurezza e licenza come dettagli finali.

3.8 Esercizi A-F

- **A — osserva:** etichetta cinque affermazioni come misura, simulazione o aspettativa.
- **B — modifica:** trasforma “il modello X è migliore” in un'ipotesi verificabile.
- **C — crea:** prepara un manifest di evidenza per una prova riproducibile.
- **D — diagnostica:** trova tre variabili di confondimento in un confronto dato.
- **E — mini-progetto:** confronta una baseline deterministica e un chatbot su dieci esempi.
- **F — prodotto:** definisci protocollo, rischi, criteri di arresto e report per il capstone annuale.

3.9 Laboratorio

Compila la diagnostica iniziale e il template docs/course/templates/evidence-manifest.json. Esegui `python3 labs/course_lab.py evidence` e completa i campi mancanti. Non serve ancora installare un modello: lo scopo è imparare a registrare una prova prima di essere affascinati dall'output.

3.10 Verifica rapida

1. Qual è la differenza fra modello e applicazione?
2. Perché una baseline deve precedere il confronto?
3. Una stima di memoria calcolata su carta è misura o simulazione?
4. Che cosa devi mantenere costante per confrontare due quantizzazioni?

Superamento: almeno 3 risposte corrette e un manifest senza ambiguità tra osservato e atteso.

3.11 Sintesi inclusiva

Un LLM genera continuazioni probabili; l'applicazione decide come usarlo. Prima di giudicare un risultato fissa domanda, confronto e misura. Proteggi i dati e limita l'autonomia in base al rischio. Se un compagno non può ripetere la prova, manca ancora un pezzo dell'evidenza.

3.12 Fonti e collegamenti

- [NIST AI Risk Management Framework](#)
- [Mappa curricolare](#)
- [Manifest di evidenza](#)
- Activity: llm-activity-m00-baseline

Capitolo 4

M01 — Mappa dell'ecosistema

Domanda guida: dove sono modello, dati e calcolo quando usiamo un assistente AI? **Durata:** 2 ore Practitioner; 4 ore AI Engineer. **Prerequisiti:** M00.

4.1 Obiettivi osservabili

Saprai distinguere modello di base, modello post-addestrato, tokenizer, runtime, API e applicazione; seguire il percorso dei dati in uno scenario locale o cloud; motivare una scelta con privacy, capacità, costo e manutenzione. Il livello AI Engineer aggiunge deployment ibrido, confini di fiducia e dipendenze operative.

4.2 Problema iniziale

Scrivi una domanda in un'interfaccia e compare una risposta. Dove è stato eseguito il calcolo? Chi conserva il prompt? L'applicazione ha consultato documenti o strumenti? Dal solo schermo non si può sapere. Per ragionare bene serve una mappa a strati.

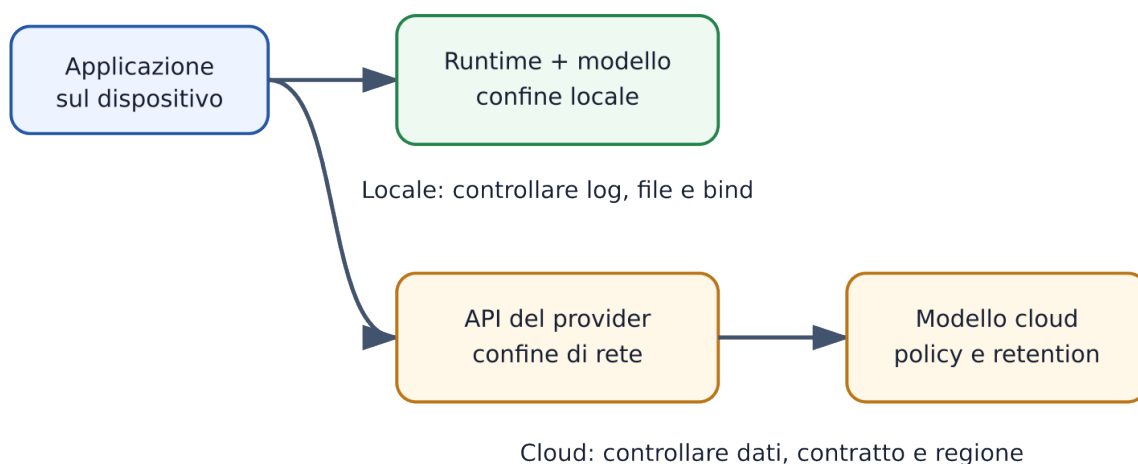
4.3 Teoria Practitioner

Il **tokenizer** converte testo o byte in identificatori. Il **modello** contiene architettura e parametri appresi. Un **checkpoint** è una revisione concreta dei pesi. Il **runtime** legge il formato dei pesi, alloca memoria ed esegue i kernel. Un **server di inferenza** espone richieste concorrenti tramite API. L'**applicazione** gestisce utenti, prompt, cronologia, retrieval e interfaccia.

“Open weight” significa che i pesi sono ottenibili secondo una licenza; non implica automaticamente codice, dati di training o libertà d'uso illimitata. “Open source” va verificato componente per componente. Un servizio cloud può usare un modello proprietario oppure ospitare un modello open weight: posizione del calcolo e regime dei pesi sono assi diversi.

Apri [Dove viaggia il prompt?](#) Nel percorso locale, prompt e pesi possono restare sulla macchina, ma installazione, download e log vanno comunque controllati. Nel cloud, la macchina invia una richiesta a un servizio soggetto a condizioni, retention e regione. “Locale” non equivale a “automaticamente sicuro”; “cloud” non equivale a “automaticamente insicuro”.

Dove viaggia il prompt?



4.4 Esempio minimo

In Ollama, un nome come `famiglia:tag` seleziona un artefatto gestito dal runtime. L'interfaccia web può inviare il prompt all'API locale `localhost`. Se la stessa interfaccia usa invece un endpoint esterno, cambia il confine di fiducia anche se l'aspetto resta identico. Documenta separatamente UI, API, runtime, modello, revisione e posizione dei dati.

4.5 Esempio realistico

Un chatbot scolastico deve rispondere su circolari. Architettura A: modello cloud e documenti inviati al provider. B: modello e indice locali. C: retrieval locale, testo anonimizzato e modello cloud. A può offrire capacità superiori; B controllo e offline; C un compromesso. La decisione include qualità, dati, latenza, costi, aggiornamenti e audit.

4.6 Livello AI Engineer: confini e deployment

Disegna data plane e control plane. Il data plane tratta prompt, embedding e output; il control plane gestisce configurazione, modelli, autorizzazioni e osservabilità. Identifica asset, attori, ingressi e trust boundary. Un processo locale con accesso a tutti i file può essere più rischioso di un'API cloud limitata a testo anonimizzato.

Nel deployment ibrido puoi usare routing per sensibilità, capacità o costo: classificazione locale e richiesta remota solo quando consentito; fallback locale senza rete; modelli differenti per task. Il routing deve essere misurato e protetto: se invia per errore dati sensibili al ramo cloud, l'architettura fallisce anche se i singoli modelli funzionano.

4.7 Confronto tra soluzioni

Criterio	Locale	Cloud	Ibrido
Avvio	download e configurazione	credenziali/API	entrambi
Privacy	controllo locale da verificare	dipende da contratto e flusso	dipende dal routing
Capacità	limitata dall'hardware	sistemi grandi disponibili	selettiva
Costi	hardware ed energia	consumo e abbonamenti	più complessi
Offline	possibile	normalmente no	parziale
Manutenzione	a carico dell'utente	in parte del provider	doppia superficie

4.8 Errori frequenti

- Confondere interfaccia e modello sottostante.
- Credere che un tag mobile identifichi per sempre gli stessi pesi.
- Ignorare log, cache, telemetria e backup nel percorso dei dati.
- Valutare soltanto il costo per token e non quello operativo.
- Dichiarare “open source” senza leggere la licenza.

4.9 Esercizi A-F

- **A:** associa dieci termini allo strato corretto.
- **B:** modifica un diagramma cloud trasformandolo in locale.
- **C:** disegna il data journey di un'app che usi davvero.
- **D:** trova il punto in cui un dato riservato supera un confine non dichiarato.
- **E:** progetta tre varianti locale/cloud/ibrida e scegli con una matrice pesata.
- **F:** realizza un router con policy, audit e fallback dimostrabile.

4.10 Laboratorio

Usa la visuale, poi completa una scheda con componenti, proprietario, posizione e dati trattati. Esegui `python3 labs/course_lab.py local-cloud` e confronta la tua classificazione. La consegna non chiede quale soluzione sia “migliore” in assoluto, ma quale soddisfi i vincoli espliciti.

4.11 Verifica rapida

Spiega in 90 secondi il percorso di un prompt locale; indica due componenti che non sono il modello; descrivi un rischio locale e uno cloud; chiarisci perché open weight e locale non sono sinonimi.

4.12 Sintesi inclusiva

L'esperienza “scrivo e ricevo una risposta” nasconde una catena. Separare tokenizer, pesi, runtime, server e applicazione rende visibili costi, responsabilità e dati. La scelta corretta dipende dal compito e dai vincoli.

4.13 Fonti e collegamenti

- [Visuale locale/cloud](#)
- [Catalogo modelli datato](#)
- Activity: `llm-activity-m01-ecosystem-map`

Capitolo 5

M02 — Predire il simbolo successivo

Domanda guida: come nasce un testo lungo da una sola previsione alla volta? **Durata:** 3 ore Practitioner; 8 ore AI Engineer. **Prerequisiti:** M00-M01; percentuali e logaritmi per l'estensione.

5.1 Obiettivi osservabili

Saprai leggere una distribuzione di probabilità, distinguere logits e probabilità, simulare la generazione autoregressiva e spiegare perché plausibilità non significa verità. Il livello AI Engineer calcola softmax, cross-entropy, entropia e perplexity e collega la previsione ai bit di un codificatore aritmetico.

5.2 Problema iniziale

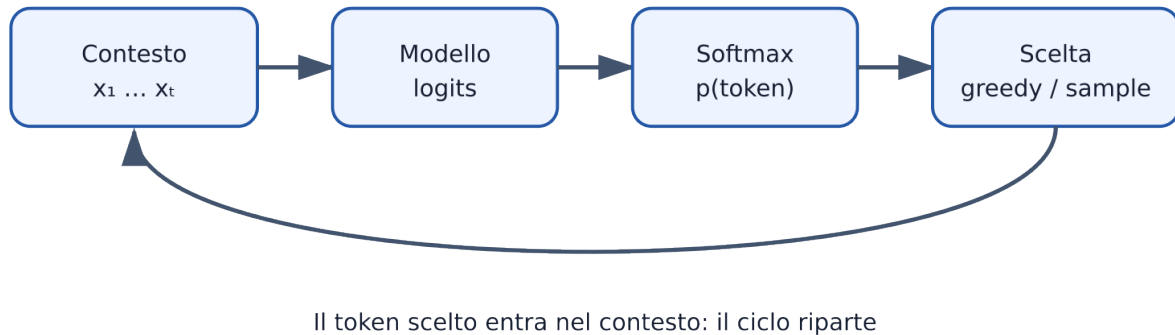
Completa “La capitale d'Italia è ...”. Una risposta sembra richiedere conoscenza geografica; per il modello l'operazione immediata è assegnare punteggi ai token possibili. Ripetendo scelta e reinserimento del token nel contesto emerge un paragrafo. Il comportamento complesso nasce da un ciclo semplice, ma i parametri che producono i punteggi hanno appreso strutture molto ricche.

5.3 Teoria Practitioner

I **logits** sono punteggi non normalizzati. La softmax li trasforma in valori positivi che sommano a uno. Il decoder sceglie un token: il massimo produce una scelta greedy; il campionamento tratta la distribuzione come una lotteria controllata. Il token scelto viene aggiunto al contesto e il modello calcola una nuova distribuzione.

Apri [il ciclo next-token](#). Cambia il contesto e osserva che non stai interrogando un archivio di frasi: stai cambiando la distribuzione condizionata. Una sequenza può essere grammaticalmente probabile e fattualmente falsa; l'obiettivo di training non contiene un verificatore universale della realtà.

Una previsione alla volta



5.4 Esempio minimo

Supponi tre candidati con probabilità mare=0,50, monte=0,30, casa=0,20. Greedy sceglie sempre “mare”. Campionando, “mare” compare circa metà delle volte su molte prove, non necessariamente cinque volte su dieci. Dopo la scelta, le probabilità del passo successivo cambiano. La probabilità dell'intera sequenza è il prodotto delle probabilità condizionate dei singoli passi.

5.5 Esempio realistico

Un modello deve produrre JSON. Anche se ogni token più probabile sembra sensato, basta una parentesi mancante per rendere invalido l'oggetto. Per questo un'applicazione robusta combina prompt, output strutturato o grammatica, validazione e retry controllato. La previsione next-token resta il motore, ma il prodotto richiede controlli esterni.

5.6 Livello AI Engineer: matematica

Per logits z_i e temperatura $T > 0$:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}.$$

Sottrarre $\max_j z_j$ prima dell'esponenziale evita overflow senza cambiare il risultato. Con target corretto y , la negative log-likelihood è $-\log p_y$; la cross-entropy media su N token è

$$L = -\frac{1}{N} \sum_{t=1}^N \log p(x_t | x_{<t}).$$

La perplexity è $\exp(L)$ quando si usano logaritmi naturali. È interpretabile come dimensione efficace dell'incertezza, ma confronti validi richiedono stesso dataset, stessa tokenizzazione e stessa convenzione. L'entropia $H(p) = -\sum_i p_i \log_2 p_i$ misura l'incertezza in bit.

5.7 Dalle probabilità ai bit

Un buon modello assegna alta probabilità al simbolo osservato. Un codificatore aritmetico può usare quelle probabilità per restringere un intervallo e rappresentare la sequenza con circa $-\log_2 p(x)$ bit. Apri [probabilità → bit](#). Per una ricostruzione lossless, encoder e decoder devono riprodurre esattamente la stessa distribuzione a ogni passo: una piccola divergenza può corrompere tutto il resto.

5.8 Errori frequenti

- Leggere probabilità come percentuale di verità.
- Sommare probabilità dei passi invece di moltiplicarle.
- Confrontare perplexity con tokenizer differenti.
- Credere che temperatura zero modifichi i pesi.
- Usare un output convincente come prova di una fonte consultata.

5.9 Esercizi A-F

- **A:** scegli il token greedy in cinque distribuzioni.
- **B:** modifica una distribuzione e prevedi come cambia l'entropia.
- **C:** implementa softmax stabile e verifica che la somma sia uno.
- **D:** correggi un calcolo di perplexity con log e base incoerenti.
- **E:** costruisci un generatore bigram e confronta strategie.
- **F:** collega un modello causale a un codec aritmetico con round trip esatto.

5.10 Laboratorio

Usa la visuale e poi esegui `python3 labs/course_lab.py next-token`. Registra distribuzione, scelta, sorpresa $-\log_2 p$ e sequenza. Per Pollicino esegui anche `python3 labs/course_lab.py arithmetic-codec` e verifica che input e output coincidano.

5.11 Verifica rapida

1. Che differenza c'è tra logit e probabilità?
2. Perché il contesto cambia a ogni token generato?
3. Perché una frase probabile può essere falsa?
4. Quale condizione rende possibile il decoding lossless?

5.12 Sintesi inclusiva

Il modello sceglie un seguito una volta alla volta. I punteggi diventano probabilità, la strategia decide il token e il ciclo riparte. La probabilità descrive la previsione del modello, non certifica la realtà. La stessa distribuzione può guidare generazione o compressione.

5.13 Fonti e collegamenti

- Claude Shannon, A Mathematical Theory of Communication (1948)
- [Visuale next-token](#)
- [Percorso Pollicino](#)
- Activity: `llm-activity-m02-next-token`

Capitolo 6

M03 — Token, byte ed embedding

Domanda guida: che cosa vede davvero un modello quando scriviamo una frase o gli diamo un file? **Durata:** 3 ore Practitioner; 10 ore AI Engineer. **Prerequisiti:** M02; vettori per l'estensione.

6.1 Obiettivi osservabili

Saprai descrivere la catena testo → byte → token → ID → embedding, verificare un round trip e spiegare perché costo e context window dipendono dai token. Il livello AI Engineer implementa una tokenizzazione elementare, una lookup table di embedding e analizza vantaggi e limiti di modelli token-, byte- e character-level.

6.2 Problema iniziale

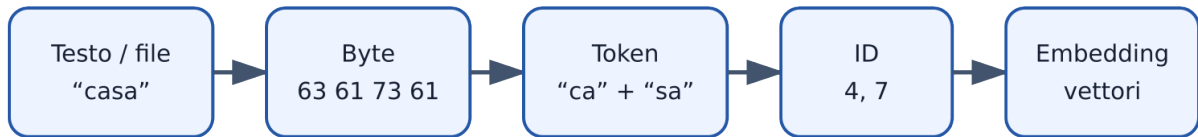
Le parole “casa”, “cassa”, un'emoji e un frammento binario non hanno la stessa rappresentazione. Un modello non riceve direttamente significati: riceve numeri costruiti da una convenzione. Cambiare tokenizer può cambiare lunghezza, costo, segmentazione delle lingue e compatibilità con i pesi.

6.3 Teoria Practitioner

Il testo Unicode viene serializzato in byte, spesso UTF-8. Un tokenizer raggruppa byte o caratteri in unità ricorrenti; ogni token ha un ID nel vocabolario. L'ID non esprime una distanza semantica: è un indice. La matrice di embedding associa l'ID a un vettore appreso. Dopo i layer, il modello produce logits sul vocabolario e il decoder riconverte gli ID in byte e testo.

Apri [Dal testo ai numeri](#). Prova parole italiane, codice, spazi e emoji. Un token non coincide necessariamente con una parola: può essere un prefisso, uno spazio più parola, un byte o un simbolo speciale. Per questo non esiste una conversione universale da parole a token.

Che cosa vede il modello?



L'ID è un indice; il significato emerge dai vettori e dal contesto.

Per un codec lossless: `decode(encode(byte))` deve restituire gli stessi byte.

6.4 Esempio minimo

Con un vocabolario didattico {"ca": 4, "sa": 7, "ssa": 9}, "casa" può diventare [4,7] e "cassa" [4,9]. Gli ID 7 e 9 non codificano una vicinanza semantica. È la matrice $E \in \mathbb{R}^{V \times d}$ a fornire i vettori: per l'ID i , l'embedding iniziale è la riga E_i .

6.5 Esempio realistico

Devi stimare se un corpus entra nel contesto. Contare caratteri non basta. Esegui il tokenizer esatto del checkpoint, conta istruzioni, documenti, cronologia e spazio riservato all'output. Un template chat aggiunge token speciali invisibili. Se cambi famiglia di modello devi ripetere il conteggio.

6.6 Livello AI Engineer: tokenizzazione ed embedding

Metodi subword come BPE partono da unità piccole e fondono coppie frequenti; Unigram seleziona segmentazioni probabili da un vocabolario candidato. I tokenizer byte-level coprono qualunque sequenza di byte, ma una singola entità visiva può occupare più unità. I modelli byte-level eliminano un vocabolario linguistico fisso e sono interessanti per file arbitrari, ma devono elaborare sequenze più lunghe.

Una lookup di embedding equivale a moltiplicare un vettore one-hot per E , ma l'indicizzazione evita il grande vettore sparso. Gli embedding contestuali prodotti dai layer non coincidono con le righe iniziali: lo stesso token assume rappresentazioni diverse in contesti diversi.

Per un file, round trip significa `decode(encode(x)) == x`. Normalizzazioni Unicode o sostituzioni di caratteri invalidi possono rompere l'uguaglianza. Per Pollicino la sequenza di byte originale è l'autorità: il percorso non deve trasformarla silenziosamente in testo.

6.7 Confronto tra rappresentazioni

Unità	Vantaggio	Costo o limite
Parola	sequenza breve	vocabolario enorme, parole ignote
Subword	buon compromesso	segmentazione dipendente dal corpus
Carattere	semplice da spiegare	Unicode e sequenze più lunghe
Byte	copertura universale e round trip	più passi da elaborare

6.8 Errori frequenti

- Chiamare token ogni parola separata da spazi.
- Interpretare l'ID come valore semantico.
- Usare il tokenizer di un modello con i pesi di un altro.
- Dimenticare token speciali e chat template nel budget.
- Normalizzare un file quando serve ricostruzione esatta.

6.9 Esercizi A-F

- **A:** segmenta manualmente una frase con un vocabolario dato.
- **B:** cambia una fusione BPE e osserva la lunghezza.
- **C:** implementa encode/decode per un tokenizer didattico.
- **D:** trova perché un round trip Unicode non coincide.
- **E:** misura token per italiano, inglese e codice su due tokenizer.
- **F:** progetta una rappresentazione byte-level con test esaustivi.

6.10 Laboratorio

Esegui `python3 labs/course_lab.py bytes` e prova file con zero byte, UTF-8 multibyte e dati non testuali. Se disponi di un tokenizer reale, registra nome e revisione e confronta rapporto byte/token su tre domini.

6.11 Verifica rapida

Disegna la catena completa; spiega ID contro embedding; mostra un caso in cui token e parola non coincidono; indica perché Pollicino preferisce preservare i byte.

6.12 Sintesi inclusiva

Il modello riceve indici, non parole. Il tokenizer stabilisce come il contenuto diventa una sequenza; l'embedding trasforma gli indici in vettori appresi. Formato, costi e limiti dipendono da questa scelta. Nei file lossless, nessun passaggio può perdere informazione.

6.13 Fonti e collegamenti

- [SentencePiece](#)
- [ByT5](#)
- [Visuale token/byte/embedding](#)
- Activity: `llm-activity-m03-token-inspector`

Capitolo 7

M04 — Apprendere dai dati

Domanda guida: come cambiano miliardi di numeri affinché la previsione migliori? **Durata:** 4 ore Practitioner; 12 ore AI Engineer. **Prerequisiti:** M02-M03; derivate e algebra lineare per l'estensione.

7.1 Obiettivi osservabili

Saprai descrivere training, validation e inferenza; interpretare una curva di loss; riconoscere overfitting, leakage e distribuzione fuori dominio. Il livello AI Engineer deriva il gradiente di un classificatore semplice, implementa un training loop e spiega optimizer, batch, learning rate e checkpoint.

7.2 Problema iniziale

Un modello memorizza perfettamente gli esempi di allenamento ma fallisce su frasi nuove. Ha ridotto la loss di training, ma non ha dimostrato di generalizzare. L'obiettivo non è ricordare il foglio delle risposte: è estrarre regolarità utili su dati non visti.

7.3 Teoria Practitioner

Nel pre-training mostriamo sequenze e chiediamo di prevedere il token successivo. La loss misura quanto il modello ha penalizzato il token osservato. La retropropagazione attribuisce una parte dell'errore ai parametri; l'optimizer li aggiorna. Un **batch** contiene più esempi prima di un aggiornamento. Un'**epoca** attraversa una volta il dataset, concetto meno netto nei grandi stream.

Separiamo training, validation e test. Il training modifica i pesi; la validation guida decisioni come quando fermarsi; il test dovrebbe essere usato alla fine. Se esempi o duplicati attraversano le separazioni, otteniamo leakage e una stima ottimistica.

7.4 Esempio minimo

Un modello con un solo parametro produce $\hat{y} = wx$. Per esempi $(1, 2)$ e $(2, 4)$, $w = 1$ sottostima. La loss quadratica segnala l'errore; il gradiente indica la direzione in cui cambiare w . Aggiornando più volte, w si avvicina a 2. Il principio è lo stesso nei Transformer, ma con moltissimi parametri, operazioni e dati.

7.5 Esempio realistico

Alleni un classificatore di messaggi scolastici. La loss di training scende sempre; quella di validation scende e poi risale. Il modello si adatta a dettagli non utili sui nuovi esempi. Puoi fermarti al checkpoint migliore, aumentare dati, regolarizzare o ridurre capacità. Prima controlla duplicati e distribuzione: non ogni curva strana è overfitting.

7.6 Livello AI Engineer: gradienti e ottimizzazione

Per logits $z = Wx$ e target one-hot y , con softmax p , la cross-entropy ha gradiente $\partial L / \partial z = p - y$. La chain rule propaga il segnale attraverso layer e operazioni. L'aggiornamento base è

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L,$$

dove η è il learning rate. Adam conserva stime mobili del primo e secondo momento del gradiente; weight decay e clipping affrontano problemi diversi e non sono sinonimi.

Con mixed precision alcune operazioni usano formati ridotti per velocità e memoria, mentre scale o copie selezionate preservano stabilità. Gradient accumulation simula batch effettivi maggiori. Un checkpoint riprendibile include pesi, optimizer, scheduler e stato casuale.

7.7 Come leggere le curve

- training e validation scendono: apprendimento compatibile con i dati;
- training scende, validation sale: possibile overfitting o shift;
- entrambe piatte: controllare learning rate, dati, implementazione e capacità;
- spike o NaN: instabilità numerica, batch anomalo o overflow;
- test sorprendentemente migliore: controllare campione, leakage e difficoltà.

7.8 Errori frequenti

- Usare il test set per scegliere iperparametri.
- Concludere dall'unico numero finale senza guardare le curve.
- Confondere una loss minore con "più verità".
- Non fissare seed e versioni durante un confronto.
- Riprendere soltanto i pesi perdendo lo stato dell'optimizer.

7.9 Esercizi A-F

- **A:** ordina forward, loss, backward e update.
- **B:** modifica il learning rate in una simulazione e descrivi la curva.
- **C:** implementa regressione o bigram model con un training loop.
- **D:** diagnostica leakage e overfitting in quattro scenari.
- **E:** confronta optimizer o batch size mantenendo costante il budget.
- **F:** addestra un piccolo LM, salva checkpoint riprendibile e redigi model card.

7.10 Laboratorio

Esegui `python3 labs/course_lab.py` loss e calcola cross-entropy e perplexity. Traccia training e validation per un modello giocattolo; salva metriche a ogni epoca e seleziona il checkpoint con una regola definita prima.

7.11 Verifica rapida

Spiega chi modifica i pesi; distingui validation e test; interpreta una curva divergente; scrivi l'aggiornamento del gradient descent e chiarisci il ruolo del learning rate.

7.12 Sintesi inclusiva

Il training confronta previsione e dato, misura l'errore e modifica i parametri. Una loss bassa sul training non basta: serve generalizzazione su dati separati. Curve, split e registrazione completa proteggono da conclusioni ingannevoli.

7.13 Fonti e collegamenti

- [Deep Learning, Goodfellow, Bengio e Courville](#)
- [Scaling Laws for Neural Language Models](#)
- Activity: `llm-activity-m04-learning-curve`

Capitolo 8

M05 — Attention e Transformer

Domanda guida: come decide ogni token quali parti del contesto usare? **Durata:** 5 ore Practitioner; 16 ore AI Engineer. **Prerequisiti:** M03-M04; matrici e softmax per l'estensione.

8.1 Obiettivi osservabili

Saprai seguire embedding → attention causale → residual/norm → MLP → logits; distinguere Query, Key e Value; spiegare maschera causale e multi-head. Il livello AI Engineer implementa un blocco decoder, controlla shape, stabilità e causalità e ne stima il costo.

8.2 Problema iniziale

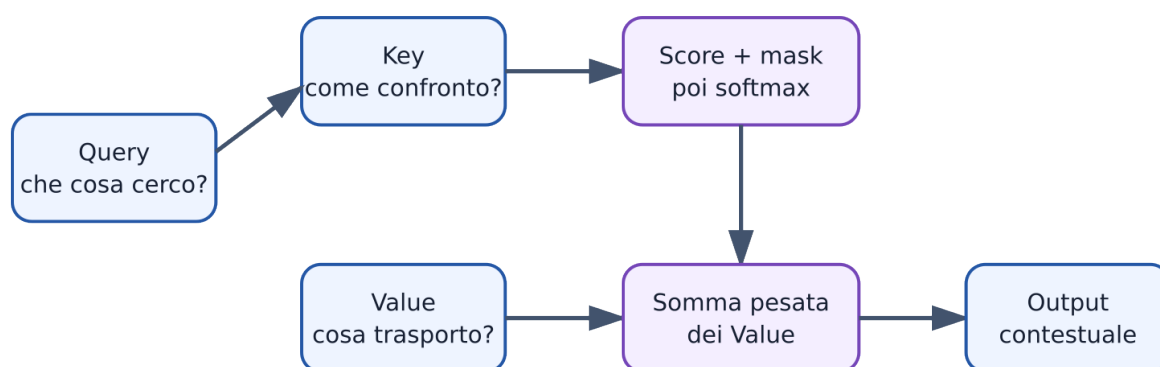
Nella frase “Maria mise il libro nello zaino perché **esso** era pesante”, per interpretare “esso” bisogna usare parti precedenti del contesto. Un Transformer non sposta un cursore simbolico: costruisce, per ogni posizione e head, pesi con cui mescolare informazioni provenienti da altre posizioni consentite.

8.3 Teoria Practitioner

Ogni token produce una **Query**, ciò che cerca; una **Key**, ciò con cui può essere confrontato; un **Value**, l'informazione da trasferire. Query e Key generano punteggi; la softmax li normalizza; la somma pesata dei Value produce un nuovo vettore. La maschera causale impedisce di leggere token futuri durante la previsione.

Apri [Attention Q/K/V](#). Cambia Query e disattiva temporaneamente la maschera. Un peso alto descrive una relazione interna di quello specifico head e layer, non una spiegazione universale del ragionamento.

Attention: cercare, confrontare, raccogliere



La maschera causale impedisce di usare il futuro.

La multi-head attention esegue più proiezioni in parallelo. Un residual conserva una strada diretta mentre aggiunge la trasformazione; la normalizzazione controlla la scala; la MLP trasforma separatamente ogni posizione. Ripetere il blocco crea rappresentazioni contestuali profonde.

8.4 Esempio minimo

Tre token hanno Key bidimensionali. La Query del terzo token è più simile alla Key del primo: dopo softmax il primo riceve peso maggiore. L'output non copia necessariamente il primo token; combina i suoi Value. Se il primo Value cambia lasciando uguali le Key, i pesi restano uguali ma cambia l'informazione trasmessa.

8.5 Esempio realistico

Nel completamento di codice, un token può usare definizioni di variabili precedenti. Con un contesto lungo, l'attention piena confronta molte coppie e consuma memoria. Runtime e architetture moderne ottimizzano cache e kernel, ma una context window dichiarata non garantisce uguale qualità a qualunque distanza.

8.6 Livello AI Engineer: equazioni e shape

Per input $X \in \mathbb{R}^{T \times d}$:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right), \quad Y = AV.$$

La maschera $M_{ij} = -\infty$ per $j > i$. Ogni riga di A deve sommare a uno. La divisione per $\sqrt{d_k}$ evita che prodotti scalari grandi saturino la softmax. In implementazione si usa un valore molto negativo compatibile con il dtype e una softmax stabile.

Le proiezioni costano circa $O(Td^2)$; score e combinazione $O(T^2d)$. Durante decoding, la KV cache evita di ricalcolare Key e Value del prefisso, ma cresce approssimativamente in modo lineare con token, layer, head KV e dimensione head.

8.7 Confronto tra implementazioni

Una versione didattica materializza la matrice $T \times T$ ed è leggibile. Kernel come FlashAttention riorganizzano il calcolo in blocchi per ridurre traffico di memoria senza cambiare la funzione matematica nei limiti numerici. Prima si verifica la reference implementation; poi si misura l'ottimizzazione.

8.8 Errori frequenti

- Scambiare Key e Value.
- Dimenticare la maschera o applicarla dopo la softmax.
- Interpretare l'attention come spiegazione causale completa.
- Ignorare batch, head e ordine delle dimensioni.
- Testare solo shape senza verificare che il futuro non influenzi il passato.

8.9 Esercizi A-F

- **A:** calcola a mano pesi su tre token.
- **B:** cambia una Query e prevedi il peso dominante.
- **C:** implementa scaled dot-product attention.
- **D:** trova una maschera applicata sull'asse sbagliato.
- **E:** confronta reference e API framework con test numerici.
- **F:** implementa forward/backward di un piccolo decoder e profila memoria.

8.10 Laboratorio

Usa la visuale, quindi costruisci matrici minuscole in NumPy o nel framework scelto. Test obbligatori: righe softmax pari a 1, nessun NaN, output finite e **causal invariance**: modificare un token futuro non deve alterare output precedenti.

8.11 Verifica rapida

Disegna Q/K/V senza analogie ambigue; calcola una riga di attention; spiega il ruolo di maschera, residual e MLP; indica un limite interpretativo dei pesi.

8.12 Sintesi inclusiva

La Query cerca, la Key permette il confronto, il Value porta informazione. La maschera protegge il futuro. Il Transformer alterna comunicazione fra posizioni e trasformazione locale, mantenendo percorsi residuali. Una figura utile deve sempre mostrare anche shape e direzione temporale.

8.13 Fonti e collegamenti

- [Attention Is All You Need](#)
- [FlashAttention](#)
- [Visuale Q/K/V](#)
- Activity: llm-activity-m05-attention

Capitolo 9

M06 — Architetture moderne

Domanda guida: quali modifiche rendono i Transformer più efficienti o capaci? **Durata:** 4 ore Practitioner; 12 ore AI Engineer. **Prerequisiti:** M05.

9.1 Obiettivi osservabili

Saprai riconoscere decoder-only, encoder-only ed encoder-decoder; spiegare intuitivamente RoPE, RMSNorm, SwiGLU, MQA/GQA, MoE e long-context. Il livello AI Engineer collega ogni tecnica al collo di bottiglia che affronta e verifica costi, compatibilità e trade-off.

9.2 Problema iniziale

Due modelli con lo stesso numero di parametri possono richiedere memoria diversa e comportarsi diversamente sul contesto. Il nome “Transformer” descrive una famiglia: dettagli su posizione, normalizzazione, head KV, attivazione e routing cambiano training e inferenza.

9.3 Teoria Practitioner

Un **encoder** rappresenta l'intero input ed è adatto a comprensione; un **decoder causale** genera da sinistra a destra; un **encoder-decoder** separa lettura e generazione. Molti LLM conversazionali moderni sono decoder-only, ma non è una legge universale.

RoPE inserisce la posizione ruotando coppie di componenti di Query e Key, così il confronto incorpora distanza relativa. **RMSNorm** normalizza la scala senza sottrarre la media. **SwiGLU** usa un ramo come gate di un altro. Queste tecniche modificano il blocco, non l'obiettivo next-token.

Nella multi-head attention classica ogni head ha Key e Value propri. **MQA** condivide un solo gruppo KV; **GQA** usa un numero intermedio di gruppi. La visuale **MHA, GQA e MQA** mostra perché meno head KV riducono la cache durante la generazione.

Un **Mixture of Experts** contiene più MLP esperte e un router attiva solo una parte per token. I parametri totali possono essere molto maggiori di quelli attivi per passo. Ciò aumenta capacità senza costo proporzionale in FLOP, ma introduce routing, comunicazione e bilanciamento.

9.4 Esempio minimo

Con 8 head Query, MHA può avere 8 gruppi KV, GQA 2 e MQA 1. Se il resto è uguale, la parte KV della cache scala con 8, 2 o 1. Non significa che l'intero modello occupi otto volte meno: i pesi e altri tensori restano.

9.5 Esempio realistico

Devi scegliere un modello locale per chat lunga. Un modello GQA quantizzato può lasciare più memoria alla KV cache rispetto a un MHA equivalente. Ma devi misurare qualità sul tuo compito, velocità del runtime e contesto effettivo; il solo limite massimo pubblicizzato non basta.

9.6 Livello AI Engineer: costi e compatibilità

Per batch B , lunghezza cache T , layer L , head KV H_{kv} , dimensione head d_h e byte s , una stima base è

$$M_{KV} \approx 2BTLH_{kv}d_h s,$$

dove 2 rappresenta Key e Value. Implementazioni, paging e quantizzazione KV cambiano il valore reale. RoPE extrapolation o rescaling può estendere la finestra nominale, ma richiede validazione su compiti sensibili alla posizione.

Nel MoE distingui parametri totali, parametri attivi, capacità del router e comunicazione fra device. Un modello può avere meno FLOP per token di un dense con pari parametri totali ma essere difficile da eseguire su una singola macchina perché tutti i pesi devono essere accessibili.

9.7 Errori frequenti

- Confrontare “parametri” senza distinguere totali e attivi.
- Concludere che long context equivalga a recupero perfetto.
- Attribuire tutta la velocità a GQA ignorando runtime e hardware.
- Applicare una tecnica di scaling RoPE non prevista dal checkpoint.
- Supporre che la stessa sigla garantisca identica implementazione.

9.8 Esercizi A-F

- **A:** riconosci i tre macro-tipi di Transformer.
- **B:** modifica il numero di head KV e aggiorna una stima.
- **C:** costruisci una scheda architetturale da una model card/config.
- **D:** trova incongruenze tra config, pesi e runtime.
- **E:** confronta due modelli su cache, qualità e latenza.
- **F:** implementa GQA o un piccolo router MoE con test di equivalenza.

9.9 Laboratorio

Apri la visuale MHA/GQA/MQA e completa una tabella con H_q , H_{kv} e memoria stimata. Verifica poi su un modello reale i campi del file di configurazione e separa dati dichiarati da valori misurati.

9.10 Verifica rapida

Associa ogni tecnica al problema affrontato; calcola il rapporto KV tra MHA e GQA; spiega perché un MoE grande non attiva tutti i parametri; indica un rischio del contesto esteso.

9.11 Sintesi inclusiva

I modelli moderni non cambiano la regola base della previsione, ma rendono calcolo, memoria e capacità più gestibili. Ogni sigla ha un beneficio, un costo e condizioni di validità: imparare a leggerli vale più che memorizzare una classifica.

9.12 Fonti e collegamenti

- [RoFormer / RoPE](#)
- [GQA](#)
- [Switch Transformers](#)
- Activity: llm-activity-m06-architectures

Capitolo 10

M07 — Pre-training, dati e scaling

Domanda guida: che cosa otteniamo aumentando dati, parametri e calcolo? **Durata:** 3 ore Practitioner; 10 ore AI Engineer. **Prerequisiti:** M04–M06.

10.1 Obiettivi osservabili

Saprai descrivere acquisizione, filtraggio, deduplicazione, mixture e data governance; interpretare le scaling law senza trasformarle in garanzie. Il livello AI Engineer ragiona su token budget, compute-optimal training, contaminazione e documentazione del dataset.

10.2 Problema iniziale

“Più dati” sembra sempre meglio. Ma duplicati, dati personali, codice con licenza incompatibile, testi tossici o test set presenti nel training possono migliorare alcune metriche e peggiorare affidabilità e legalità. Il dataset è parte del comportamento del modello.

10.3 Teoria Practitioner

Una pipeline di pre-training raccoglie fonti, estrae contenuti, filtra qualità, lingua e sicurezza, rimuove duplicati, applica pesi alle sorgenti e tokenizza. Ogni filtro produce falsi positivi e falsi negativi. La mixture decide quanto spesso il modello vede ciascun dominio; una piccola fonte può essere sovracampionata.

Le **scaling law** descrivono regolarità empiriche: entro un intervallo, la loss tende a migliorare in modo prevedibile aumentando parametri, dati e compute. Non dicono che ogni capacità cresca uniformemente né risolvono qualità, allineamento o contaminazione.

10.4 Esempio minimo

Un corpus contiene cento copie della stessa pagina e cento pagine diverse. Contare i documenti suggerisce 200 esempi; deduplicare rivela solo 101 contenuti. Se il test contiene la pagina duplicata, il punteggio può misurare memoria invece di generalizzazione.

10.5 Esempio realistico

Per un modello didattico italiano costruisci una data card: origine, autorizzazione, periodo, lingue, rimozione PII, deduplicazione, split e limiti. Prima del training calcola hash dei documenti e cerca sovrapposizioni tra train e test. Conserva lo script di trasformazione, non soltanto il dataset finale.

10.6 Livello AI Engineer: budget e contaminazione

Il compute di training di un decoder dense è spesso stimato come ordine di grandezza $C \approx 6ND$, con N parametri e D token, ma architettura e implementazione cambiano la costante. Risultati compute-optimal mostrano che, dato un budget, un modello troppo grande e poco addestrato può essere peggiore di uno più piccolo con più token.

La contaminazione non è soltanto corrispondenza esatta. Parafrasi, soluzioni, traduzioni e dati derivati possono attraversare gli split. Si usano hashing, MinHash o similarità embedding, ma nessun filtro prova assenza completa. I benchmark devono dichiarare cutoff temporale e procedure di decontaminazione.

Data governance comprende base giuridica, consenso o licenza, diritto di rimozione, provenienza, sicurezza e impatto sui gruppi. “Disponibile sul web” non significa automaticamente riutilizzabile per training o redistribuzione.

10.7 Errori frequenti

- Contare volume grezzo ignorando duplicati.
- Usare benchmark pubblici durante molte iterazioni e chiamarli ancora test.
- Concludere che una legge empirica valga fuori dal regime osservato.
- Documentare le fonti ma non le trasformazioni.
- Confondere accessibilità con licenza.

10.8 Esercizi A-F

- **A:** ordina gli stadi di una pipeline dati.
- **B:** applica deduplicazione a un piccolo corpus.
- **C:** redigi una data card con provenienza e limiti.
- **D:** individua leakage tra train, validation e test.
- **E:** progetta una mixture multilingue e giustifica i pesi.
- **F:** costruisci pipeline versionata con audit, decontaminazione e report.

10.9 Laboratorio

Esegui `python3 labs/course_lab.py scaling` per esplorare una relazione semplificata. Poi crea un corpus giocattolo, calcola hash, elimina duplicati e mostra come cambia una metrica. L'obiettivo è vedere quanto il dataset possa alterare una conclusione.

10.10 Verifica rapida

Spiega perché più dati non equivale a dati migliori; distingui scaling law e garanzia; descrivi due forme di contaminazione; elenca i campi minimi di una data card.

10.11 Sintesi inclusiva

Il modello apprende ciò che la pipeline rende frequente e osservabile. Dimensione, qualità, mixture, licenze e contaminazione devono essere trattate insieme. Le scaling law aiutano a pianificare, non sostituiscono la misura.

10.12 Fonti e collegamenti

- [Scaling Laws for Neural Language Models](#)
- [Training Compute-Optimal Large Language Models](#)
- [Data Statements for NLP](#)

- Activity: llm-activity-m07-data-card

Capitolo 11

M08 — Post-training e reasoning

Domanda guida: come diventa assistente un modello che ha imparato soprattutto a continuare testi? **Durata:** 3 ore Practitioner; 12 ore AI Engineer. **Prerequisiti:** M04–M07.

11.1 Obiettivi osservabili

Saprai distinguere pre-training, supervised fine-tuning, preference optimization, RL e distillazione; riconoscere quando più token di ragionamento aiutano o sprecano risorse. Il livello AI Engineer formula gli obiettivi principali e progetta confronti controllati tra policy.

11.2 Problema iniziale

Un modello base può completare “Domanda: ... Risposta: ...”, ma non necessariamente seguire bene istruzioni o rifiutare richieste rischiose. Per renderlo un assistente si aggiungono esempi, preferenze e feedback. Questo migliora il comportamento osservabile, senza trasformare il modello in un oracolo.

11.3 Teoria Practitioner

Nel **supervised fine-tuning (SFT)** il modello imita risposte curate. Nei metodi di **preference optimization** impara a favorire una risposta scelta rispetto a una rifiutata. Nell'**RLHF** un segnale derivato da preferenze guida una policy con reinforcement learning. Varianti possono usare feedback umano, AI o verificatori automatici.

“Reasoning model” descrive sistemi addestrati o configurati per spendere calcolo aggiuntivo prima della risposta, usare tracce interne, strumenti o verifiche. Una risposta più lunga non prova un ragionamento migliore. Il test deve misurare risultato, robustezza, costo e capacità di correggersi.

11.4 Esempio minimo

Prompt: “Rispondi con un numero”. Il modello base continua con spiegazioni; dopo SFT rispetta più spesso il formato. Una coppia di preferenze può insegnare a privilegiare la risposta corretta e concisa. Tuttavia, se le preferenze premiano stile sicuro invece di correttezza, il modello può imparare sicurezza apparente.

11.5 Esempio realistico

Per problemi matematici confronta risposta diretta, scomposizione guidata e uso di un calcolatore. Mantieni stesso modello e dataset; registra accuratezza, token, latenza e fallimenti. Se il

calcolatore migliora l'esattezza, il merito appartiene al sistema modello+strumento, non ai soli pesi.

11.6 Livello AI Engineer: obiettivi

Nell'SFT si minimizza la negative log-likelihood dei token di risposta, spesso mascherando la parte prompt. Un preference model può stimare $r(x, y)$ da coppie (y_w, y_l) . La DPO ottimizza direttamente una probabilità relativa rispetto a una policy di riferimento; il dettaglio della parametrizzazione conta, ma l'intuizione è aumentare il margine per la risposta preferita senza allontanarsi senza controllo.

RL con reward verificabile è particolarmente utile quando il risultato può essere controllato, per esempio test di codice o esito matematico. Anche qui reward hacking e distribuzioni strette sono rischi: una policy può massimizzare il verificatore sfruttandone lacune.

Distillazione trasferisce comportamento da un teacher a uno student mediante output, logits o dati sintetici. Riduce costo di esecuzione ma può trasferire errori e non conferisce automaticamente le stesse capacità fuori distribuzione.

11.7 Errori frequenti

- Confondere instruction tuning con acquisizione di nuovi fatti garantiti.
- Valutare reasoning dalla lunghezza della spiegazione.
- Usare come giudice lo stesso modello senza controlli indipendenti.
- Ignorare il modello di riferimento o la forza della regolarizzazione.
- Premiare una metrica facilmente manipolabile.

11.8 Esercizi A-F

- **A:** classifica esempi come pre-training, SFT o preferenza.
- **B:** riscrivi una coppia di preferenza ambigua.
- **C:** costruisci un piccolo dataset SFT con criteri espliciti.
- **D:** diagnostica reward hacking in un verificatore.
- **E:** confronta tre strategie di reasoning con budget uguale.
- **F:** implementa un esperimento SFT/DPO ridotto e valuta regressioni.

11.9 Laboratorio

Esegui `python3 labs/course_lab.py reasoning` su problemi verificabili. Pre-registra modalità e budget, poi confronta risposta diretta, scomposizione e tool. Non conservare soltanto l'accuratezza media: raccogli categorie di errore.

11.10 Verifica rapida

Distingui SFT, preferenze e RL; spiega perché il post-training non garantisce verità; proponi una metrica contro verbosity; descrivi un rischio della distillazione.

11.11 Sintesi inclusiva

Il pre-training costruisce capacità generali di previsione; il post-training orienta comportamento, formato e preferenze. Reasoning e strumenti possono migliorare compiti difficili, ma consumano risorse e devono essere verificati sul risultato, non sull'apparenza.

11.12 Fonti e collegamenti

- [InstructGPT](#)
- [Direct Preference Optimization](#)
- [Timeline dei paper](#)
- Activity: llm-activity-m08-post-training

Capitolo 12

M09 — Pesi, formati e licenze

Domanda guida: che cosa stiamo realmente scaricando quando scegliamo un modello locale?

Durata: 3 ore Practitioner; 8 ore AI Engineer. **Prerequisiti:** M01 e M06.

12.1 Obiettivi osservabili

Saprai leggere una model card, distinguere architettura, checkpoint, precisione, quantizzazione, formato e licenza; scegliere un artefatto compatibile con runtime e uso. Il livello AI Engineer ispeziona metadati, shard, tensori e conversioni e costruisce una supply chain riproducibile.

12.2 Problema iniziale

Lo stesso nome di modello compare in file da dimensioni diverse: originale BF16, quantizzazioni a 8 o 4 bit, conversioni GGUF e varianti fine-tuned. Non sono intercambiabili. Una scelta sbagliata può non caricarsi, produrre output degradato o violare la licenza.

12.3 Teoria Practitioner

L'**architettura** definisce le operazioni e la forma dei tensori. Il **checkpoint** contiene valori appresi in una revisione. Un **formato contenitore** organizza tensori e metadati; `safetensors` evita deserializzazione di codice arbitrario tipica di formati più generici, mentre GGUF è progettato per ecosistemi di inferenza come llama.cpp e può includere tokenizer e metadati.

La **precisione** descrive la rappresentazione numerica, per esempio FP32, BF16 o FP16. La **quantizzazione** mappa valori in formati più compatti con scale e gruppi. Sigle come Q4 non specificano da sole algoritmo, group size o qualità.

Una licenza può consentire i pesi ma imporre condizioni su uso commerciale, ridistribuzione, utenti o derivati. Controlla testo della licenza, model card e provenienza dell'artefatto; una conversione comunitaria non eredita magicamente affidabilità.

12.4 Esempio minimo

Un modello da 7 miliardi di parametri richiede circa 14 GB solo per pesi a 2 byte, prima di cache e overhead. Una quantizzazione nominale a 4 bit suggerisce circa 3,5 GB grezzi, ma scale, metadati e allineamenti aumentano il file. La dimensione su disco non coincide esattamente con memoria residente.

12.5 Esempio realistico

Per scegliere un artefatto annota: repository, commit o digest, file, hash, architettura, tokenizer, chat template, quantizzazione, licenza, runtime minimo e fonte. Prova il caricamento offline dopo il download. Se il tag può cambiare, non è sufficiente per un esperimento riproducibile.

12.6 Livello AI Engineer: ispezione e conversione

I grandi checkpoint possono essere suddivisi in shard con un indice che mappa tensori e file. Una conversione deve preservare nomi, shape, tokenizer, token speciali, configurazione RoPE e tying dei pesi. Dopo conversione esegui test su logits o output con tolleranza dichiarata, non solo “il file si apre”.

Il formato non determina da solo il kernel: runtime diversi possono leggere lo stesso contenitore con implementazioni differenti. Distingui peso quantizzato staticamente, quantizzazione dinamica delle attivazioni e quantizzazione della KV cache. Registra tool e versione della conversione per evitare artefatti non ricostruibili.

Per la supply chain verifica hash, firma quando disponibile, identità dell'autore, dipendenze e codice remoto. Evita `trust_remote_code` senza review e sandbox. Conserva SBOM o almeno inventario di modelli e licenze.

12.7 Scheda di decisione

Campo	Domanda
Capacità	il checkpoint è adatto al task e alla lingua?
Memoria	pesi, KV cache e overhead entrano?
Runtime	architettura e quantizzazione sono supportate?
Licenza	uso e redistribuzione sono consentiti?
Provenienza	repository, revisione e hash sono affidabili?
Template	prompt e token speciali sono quelli previsti?

12.8 Errori frequenti

- Usare soltanto il numero di parametri per scegliere.
- Confondere formato file e precisione numerica.
- Trattare tutti i “4 bit” come equivalenti.
- Scaricare una conversione senza provenienza o hash.
- Ignorare chat template e tokenizer.
- Copiare nel repository libri o asset licensed usati solo come riferimento.

12.9 Esercizi A-F

- **A:** associa formato, dtype e quantizzazione alle definizioni.
- **B:** completa una model card incompleta.
- **C:** confronta tre artefatti con la scheda di decisione.
- **D:** trova incompatibilità tra config e pesi.
- **E:** converti un modello piccolo e verifica equivalenza entro tolleranza.
- **F:** costruisci pipeline firmata di acquisizione, scan, conversione e rollback.

12.10 Laboratorio

Compila `docs/course/templates/model-decision.md` per due candidati. Esegui `python3 labs/course_lab.py` memory come prima stima, quindi confronta dimensione file e memoria misurata quando il runtime sarà disponibile.

12.11 Verifica rapida

Spiega checkpoint contro architettura; formato contro quantizzazione; elenca i dati necessari a fissare un artefatto; interpreta una condizione di licenza senza sostituirti a una consulenza legale.

12.12 Sintesi inclusiva

Il nome del modello non basta. Per usare pesi locali servono artefatto preciso, tokenizer, template, formato, quantizzazione, runtime e licenza compatibili. Una scelta riproducibile è identificata da revisioni e hash, non da un'etichetta mobile.

12.13 Fonti e collegamenti

- [safetensors](#)
- [GGUF](#)
- [Inventario Manning](#)
- Activity: `llm-activity-m09-model-selection`

Capitolo 13

M10 — Hardware e quantizzazione

Domanda guida: quale modello entra davvero nella macchina e a quale velocità? **Durata:** 4 ore Practitioner; 12 ore AI Engineer. **Prerequisiti:** M06 e M09.

13.1 Obiettivi osservabili

Saprai stimare memoria di pesi e KV cache, distinguere RAM/VRAM, bandwidth e compute, misurare time-to-first-token e token/s e confrontare quantizzazioni. Il livello AI Engineer analizza roofline, batching, offload e metodi weight-only o weight-activation.

13.2 Problema iniziale

Un file da 20 GB entra in una macchina con 36 GB di memoria? Forse. Oltre ai pesi servono runtime, cache, buffer e sistema operativo; il contesto e il parallelismo cambiano il picco. “Si scarica” non significa “si esegue bene”.

13.3 Teoria Practitioner

La capacità di memoria decide se il carico è possibile. La **bandwidth** misura quanto rapidamente i dati arrivano alle unità di calcolo; i **FLOPS/TOPS** stimano operazioni, ma formato e kernel devono usarle. CPU, GPU e acceleratori hanno gerarchie e supporto differenti. Nella memoria unificata, CPU e GPU condividono lo stesso pool, ma restano pressione, bandwidth e limiti del sistema.

La quantizzazione riduce la rappresentazione dei pesi e talvolta di attivazioni o cache. Modelli più compatti possono essere più veloci e lasciare spazio al contesto, ma la perdita di qualità dipende da metodo, layer, task e runtime. Non esiste “la” qualità dei 4 bit.

Apri [Ollama e memoria](#). Separa **TTFT**, il tempo prima del primo token, da **decode throughput**, i token generati al secondo. Prefill e decode hanno colli di bottiglia diversi.

13.4 Esempio minimo

Pesi grezzi: $M_w \approx Nb/8$, con N parametri e b bit medi. Un modello 8B a 4 bit richiede circa 4 GB grezzi, non il totale reale. Aggiungi metadati, scale, buffer e KV cache. Applica margine invece di occupare il 100% della memoria.

13.5 Esempio realistico

Sul Mac M4 Pro 36 GB selezioni tre tag Ollama: piccolo, medio e più grande quantizzato. Per ciascuno registri download, memoria idle e picco, TTFT, token/s, qualità su fixture e temperatura

del sistema. La scelta per la classe privilegia affidabilità e tempi prevedibili, non il massimo numero di parametri caricabile una volta.

13.6 Livello AI Engineer: stime e roofline

La KV cache base è

$$M_{KV} \approx 2BTLH_{kv}d_h s.$$

Il fattore 2 rappresenta Key e Value. Paged attention, cache quantizzata e allocatori aggiungono dettagli. Nel decode a batch piccolo si rileggono molti pesi per produrre pochi token: spesso il limite è la bandwidth. Con batch maggiore si riusa meglio il peso ma crescono latenza e memoria.

Metodi weight-only conservano attivazioni a precisione maggiore; W8A8 quantizza anche attivazioni e richiede kernel compatibili. Group size più piccolo usa più scale e può preservare qualità, ma aumenta overhead. Per confrontare quantizzazioni usa stesso checkpoint sorgente, template, dataset e runtime.

13.7 Protocollo di misura

1. Riavvia o stabilizza lo stato e registra processi concorrenti.
2. Separa cold start da richieste successive.
3. Fissa prompt, output token, context length e seed quando disponibile.
4. Ripeti e riporta mediana e dispersione, non solo il caso migliore.
5. Registra picco di memoria, TTFT, token/s ed energia se misurabile.
6. Valuta qualità sul task reale e annota errori.

13.8 Errori frequenti

- Usare la dimensione del file come RAM esatta.
- Confrontare token/s con output o contesti diversi.
- Scambiare TTFT e velocità di decode.
- Credere che una GPU con più FLOPS sia sempre più veloce.
- Riempire tutta la memoria senza margine operativo.

13.9 Esercizi A-F

- **A:** stima memoria grezza di quattro modelli.
- **B:** cambia contesto e aggiorna la KV cache.
- **C:** costruisci un foglio di budget completo.
- **D:** trova errori in un benchmark non controllato.
- **E:** confronta due quantizzazioni su qualità e prestazioni.
- **F:** profila un serving multiutente e proponi batching/offload.

13.10 Laboratorio

Esegui `python3 labs/course_lab.py memory` e completa prima le stime. Il rehearsal reale usa `docs/course/rehearsal/README.md`: non inventare dati hardware prima dell'esecuzione. Conserva un manifest distinto per ogni artefatto.

13.11 Verifica rapida

Calcola pesi grezzi; spiega perché il totale è maggiore; distingui bandwidth e compute; motiva una quantizzazione senza dire soltanto "occupa meno".

13.12 Sintesi inclusiva

Prima di scaricare, fai un budget. Memoria abilita il modello; bandwidth, kernel e batch determinano gran parte della velocità. Quantizzare è un compromesso misurabile tra spazio, prestazioni e qualità.

13.13 Fonti e collegamenti

- [Visuale richiesta e memoria](#)
- [Visuale KV cache](#)
- [Rehearsal](#)
- Activity: `llm-activity-m10-memory-budget`

Capitolo 14

M11 — Ollama e inferenza locale

Domanda guida: come scegliamo, eseguiamo e documentiamo un modello locale? **Durata:** 4 ore Practitioner; 8 ore AI Engineer. **Prerequisiti:** M09-M10; terminale e HTTP di base.

14.1 Obiettivi osservabili

Saprai installare e verificare Ollama, acquisire un modello compatibile, usare CLI e API, fissare parametri, osservare streaming e gestire errori. Il livello AI Engineer documenta template, opzioni, lifecycle, concorrenza e riproducibilità.

14.2 Problema iniziale

Digitare `ollama run nome` produce una risposta, ma non certifica quale artefatto sia stato usato, quanta memoria richieda o se il test sia ripetibile. Il laboratorio trasforma la demo in un esperimento tracciabile.

14.3 Teoria Practitioner

Ollama gestisce modelli, avvia un server locale ed espone API. Prima verifica la documentazione corrente: comandi, tag e disponibilità cambiano. `ollama list` mostra gli artefatti locali; `ollama show` ne espone informazioni; `ollama run` apre una sessione; l'API consente a un'applicazione di inviare richieste senza simulare la tastiera.

Apri [il ciclo della richiesta](#). Il prompt passa al template chat e al tokenizer; il runtime carica i pesi, esegue prefill e poi decode; la risposta può arrivare in streaming. "localhost" limita il percorso di rete solo se bind, proxy e applicazioni sono configurati correttamente.

14.4 Esempio minimo

```
ollama --version
ollama list
ollama show MODELLO
ollama run MODELLO "Rispondi soltanto con OK"
```

Sostituisci MODELLO con un tag scelto dal catalogo **verificato il giorno della prova**. Conserva output dei primi tre comandi e il digest quando disponibile. Non inserire il nome di un modello nel corso stabile: il catalogo datato gestisce le parti volatili.

Una richiesta API tipica:

```
curl http://localhost:11434/api/chat -d '{
  "model": "MODELLO",
  "messages": [{"role": "user", "content": "Rispondi con OK"}],
```

```
"stream": false,  
"options": {"temperature": 0}  
'}
```

14.5 Esempio realistico

Costruisci un piccolo client Python con timeout, controllo dello status HTTP, parsing esplicito e log senza contenuto sensibile. Esegui una fixture di dieci prompt, registra configurazione e tempi e salva output separati. Se il server non risponde, l'app deve mostrare un errore utile; non deve bloccarsi indefinitamente.

14.6 Livello AI Engineer: template e lifecycle

L'API riceve ruoli e opzioni, ma il runtime deve serializzarli nel template previsto dal checkpoint. Un template incompatibile può degradare un ottimo modello. Ispeziona il Modelfile o i metadati e differenzia system prompt, messaggi, stop token e parametri di sampling.

Il lifecycle comprende download, verifica, caricamento, keep-alive, scaricamento e aggiornamento. Un tag mobile può cambiare: per benchmark conserva digest, data e versione Ollama. Con richieste concorrenti misura queueing e memoria; il throughput aggregato può crescere mentre la latenza per utente peggiora.

14.7 Gestione sicura degli errori

- timeout di connessione e risposta;
- modello assente o non caricabile;
- memoria insufficiente;
- risposta troncata o JSON invalido;
- stream interrotto;
- server esposto su interfacce non previste;
- aggiornamento che cambia comportamento.

Ogni errore deve produrre messaggio, codice o evidenza diagnostica senza mostrare segreti.

14.8 Errori frequenti

- Copiare un comando con modello non adatto al proprio hardware.
- Non registrare digest, versione e template.
- Assumere che temperature: 0 garantisca bitwise determinism.
- Usare retry infiniti.
- Pubblicare il server sulla LAN senza autenticazione o policy.

14.9 Esercizi A-F

- **A:** verifica servizio, elenco e metadati.
- **B:** modifica un parametro e confronta output.
- **C:** scrivi un client con timeout e validazione.
- **D:** diagnostica quattro failure case preparati.
- **E:** costruisci un'app locale con streaming e log riproducibile.
- **F:** realizza serving controllato con benchmark, policy e rollback.

14.10 Laboratorio

Segui docs/course/rehearsal/README.md quando sarà disponibile il Mac M4 Pro 36 GB. Prima del rehearsal puoi esercitarti con `python3 labs/course_lab.py ollama-request`, che costruisce

e valida una richiesta senza dichiarare esecuzione hardware.

14.11 Verifica rapida

Mostra la differenza tra CLI e API; identifica modello e runtime; dimostra timeout e gestione di un modello assente; spiega quali dati servono per ripetere la prova.

14.12 Sintesi inclusiva

Ollama rende semplice iniziare, non elimina le decisioni. Un'esecuzione seria fissa artefatto, runtime, template, parametri, hardware e fixture. L'applicazione deve gestire errori e proteggere il confine locale.

14.13 Fonti e collegamenti

- [Documentazione Ollama](#)
- [Catalogo modelli del corso](#)
- [Rehearsal Ollama](#)
- Activity: `llm-activity-m11-ollama`

Capitolo 15

M12 — Sampling e prompting

Domanda guida: come controlliamo una distribuzione senza fingere di cambiare il modello?

Durata: 3 ore Practitioner; 8 ore AI Engineer. **Prerequisiti:** M02 e M11.

15.1 Obiettivi osservabili

Saprai spiegare temperature, top-k, top-p, seed, stop e lunghezza; progettare prompt con istruzioni, dati e formato separati; validare output strutturati. Il livello AI Engineer misura entropia, calibrazione e interazioni tra decoder e grammar constraints.

15.2 Problema iniziale

Lo stesso modello produce una poesia creativa e un JSON rigoroso. Non servono necessariamente pesi diversi: prompt, template e decoder cambiano il comportamento. Ma nessun parametro garantisce da solo correttezza.

15.3 Teoria Practitioner

La **temperature** riscalda o concentra la distribuzione. **Top-k** conserva i k candidati più probabili. **Top-p** conserva il più piccolo insieme con massa cumulativa almeno p. Dopo il filtro si rinormalizza e si campiona. Seed e implementazione influenzano la ripetibilità; stop e limite token controllano la terminazione.

Apri [Sampling controls](#). Osserva che temperature, top-k e top-p interagiscono: non sono tre manopole indipendenti. Per compiti fattuali o strutturati si parte da bassa variabilità; per esplorazione creativa si può aumentarla e generare più candidati.

Un prompt robusto separa ruolo/obiettivo, input non fidato, vincoli, formato e criteri. Delimitare un documento non lo rende sicuro: il modello può seguire istruzioni contenute nei dati. L'applicazione deve validare e limitare le conseguenze.

15.4 Esempio minimo

Con probabilità [0,50, 0,25, 0,15, 0,10], top-k 2 conserva i primi due; top-p 0,70 conserva anch'esso due elementi perché raggiungono 0,75. Con distribuzioni diverse i due filtri selezionano insiemi differenti.

15.5 Esempio realistico

Vuoi estrarre nome, data e importo. Definisci schema, chiedi JSON senza testo extra, usa structured output se disponibile, valida tipi e campi, rifiuta o riprova con limite. Non inserire

direttamente l'output in SQL o in un comando. La validazione è parte della funzione applicativa.

15.6 Livello AI Engineer: decoder e vincoli

Applicare temperature ai logits precede normalmente top-k/top-p, ma dettagli del runtime possono cambiare. Repetition penalty e frequency/presence penalty non sono equivalenti e possono danneggiare codice o dati. Constrained decoding maschera token che renderebbero impossibile completare una grammatica: garantisce sintassi entro il vincolo, non verità dei valori.

Misura diversità con tasso di duplicazione o entropia e qualità con test specifici. La calibrazione confronta probabilità dichiarata e frequenza osservata; i logits degli LLM non sono automaticamente probabilità affidabili di correttezza semantica.

15.7 Errori frequenti

- Usare temperature come “livello di intelligenza”.
- Cambiare più parametri contemporaneamente.
- Credere che JSON valido sia contenuto corretto.
- Inserire dati non fidati dentro istruzioni privilegiate.
- Usare prompt segreti come unico controllo di sicurezza.

15.8 Esercizi A-F

- **A:** applica top-k e top-p a distribuzioni date.
- **B:** modifica un prompt ambiguo separando dati e istruzioni.
- **C:** crea schema e validatore per un output.
- **D:** diagnostica una combinazione che tronca sempre la risposta.
- **E:** costruisci un confronto controllato tra decoder.
- **F:** implementa constrained decoding o un gateway con policy e test avversariali.

15.9 Laboratorio

Usa la visuale e `python3 labs/course_lab.py sampling`. Con modello locale disponibile, esegui una griglia piccola cambiando una sola variabile, conserva output e valuta formato, diversità, correttezza e costo.

15.10 Verifica rapida

Calcola un insieme top-p; spiega temperature contro top-k; progetta un prompt con input non fidato; indica che cosa garantisce e non garantisce una grammatica.

15.11 Sintesi inclusiva

Il decoder sceglie dalla distribuzione prodotta dal modello. Le manopole controllano varietà e terminazione, non aggiungono conoscenza. Prompt chiari aiutano; schema, validazione e limiti rendono l'applicazione affidabile.

15.12 Fonti e collegamenti

- [The Curious Case of Neural Text Degeneration](#)
- [Visuale sampling](#)
- Activity: `llm-activity-m12-sampling`

Capitolo 16

M13 — Applicazioni conversazionali

Domanda guida: che cosa serve attorno al modello per ottenere una chat affidabile? **Durata:** 4 ore Practitioner; 10 ore AI Engineer. **Prerequisiti:** M11–M12; Python e HTTP di base.

16.1 Obiettivi osservabili

Saprai progettare messaggi, stato, streaming, limiti e gestione degli errori; costruire un client locale minimo; distinguere memoria dell'applicazione e contesto del modello. Il livello AI Engineer tratta concorrenza, idempotenza, backpressure, osservabilità e test.

16.2 Problema iniziale

Un ciclo input/output che chiama il modello sembra una chat. Dopo alcuni turni però il contesto cresce, le istruzioni si contraddicono, la rete cade o un utente invia dati sensibili. Il prodotto deve governare tutto ciò che il modello non governa.

16.3 Teoria Practitioner

Una conversazione è una sequenza di messaggi con ruolo e contenuto. L'app decide quali messaggi reinviare, quali riassume e quali elimina. Il modello non “ricorda” una sessione precedente se l'app o il servizio non gli forniscono stato. Context window e memoria persistente sono concetti distinti.

Lo streaming migliora il tempo percepito, ma richiede stati espliciti: in attesa, in ricezione, completato, annullato, errore. L'interfaccia non deve mostrare una risposta parziale come definitivamente valida. Timeout, cancel e retry devono evitare duplicazioni o richieste infinite.

16.4 Esempio minimo

```
import json
from urllib.request import Request, urlopen

payload = {"model": "MODELLO", "messages": [{"role": "user", "content": "Ciao"}], "stream": True}
request = Request("http://localhost:11434/api/chat", data=json.dumps(payload).encode(), headers={"Content-Type": "application/json"})
with urlopen(request, timeout=30) as response:
    data = json.load(response)
    print(data["message"]["content"])
```

Questo frammento è un punto di partenza, non un'app completa: mancano validazione, errori, configurazione, log sicuri e test.

16.5 Esempio realistico

Llma_Chatbot può separare adapter del provider, dominio della conversazione e UI. Lo stesso test di contratto deve funzionare con Ollama e con un mock deterministico. Un limite token arresta output eccessivi; la cronologia è ridotta con una policy esplicita; i log conservano ID e tempi, non prompt sensibili.

16.6 Livello AI Engineer: architettura

Definisci un'interfaccia del provider con request, eventi stream, usage ed errori normalizzati. Mantieni il dominio indipendente dall'SDK. Usa un correlation ID per collegare richiesta, tentativi e metriche. I retry sono sicuri soltanto quando l'operazione è idempotente o protetta da chiavi; per tool con effetti esterni serve conferma e deduplicazione.

La backpressure impedisce che produttore e consumatore saturino memoria. Con molti utenti, queueing e rate limit proteggono il runtime. L'osservabilità include TTFT, latenza totale, token, error category, cancel e modello/digest, con redazione dei contenuti.

16.7 Struttura consigliata

Componente	Responsabilità
UI	input, rendering, accessibilità, cancel
Conversation service	stato, policy del contesto, orchestrazione
Provider adapter	protocollo Ollama/cloud, streaming, errori
Validator	schema, limiti, sanitizzazione
Storage	sessioni autorizzate, retention, cifratura
Telemetry	metriche tecniche prive di segreti

16.8 Errori frequenti

- Inserire l'intera cronologia senza budget.
- Accoppiare UI direttamente a un SDK.
- Registrare prompt e chiavi nei log.
- Riprovare automaticamente un'azione con effetti.
- Confondere stop dello stream e annullamento lato server.
- Testare soltanto con il modello reale e output variabili.

16.9 Esercizi A-F

- **A:** completa una sequenza di messaggi con ruoli.
- **B:** aggiungi timeout e messaggio d'errore al client.
- **C:** implementa adapter Ollama dietro un'interfaccia.
- **D:** correggi un bug di doppio invio durante retry.
- **E:** costruisci chat locale con streaming, cancel e test mock.
- **F:** realizza servizio multiutente con rate limit, audit e benchmark.

16.10 Laboratorio

Parti dal client minimo e usa un server mock prima del modello reale. Testa risposta valida, timeout, JSON invalido, stream interrotto e cancel. Solo dopo collega Ollama e registra manifest e metriche.

16.11 Verifica rapida

Spiega dove vive la memoria; mostra come sostituire il provider; gestisci un timeout senza perdere lo stato; elenca tre metriche che non richiedono salvare il prompt.

16.12 Sintesi inclusiva

La chat è un sistema, non una casella di testo. L'applicazione possiede stato, privacy, errori, limiti e interfaccia; il modello produce continuazioni. Separare i componenti permette test, sostituzione e controllo.

16.13 Fonti e collegamenti

- [Documentazione API Ollama](#)
- [Valutazione del libro Local AI Models](#)
- Activity: llm-activity-m13-chatbot

Capitolo 17

M14 — Valutazione

Domanda guida: come sappiamo se un modello o un'applicazione è adatto al nostro scopo?

Durata: 4 ore Practitioner; 14 ore AI Engineer. **Prerequisiti:** M00, M10-M13.

17.1 Obiettivi osservabili

Saprai trasformare requisiti in dataset, metriche e soglie; confrontare qualità, latenza, memoria, costo e sicurezza; costruire un regression set. Il livello AI Engineer stima incertezza, accordo tra annotatori e limiti di model-as-judge.

17.2 Problema iniziale

Un benchmark pubblico assegna 78 punti al modello A e 75 al B. Per il tuo chatbot scolastico A è davvero migliore? Non sappiamo lingua, task, formato, hardware, contaminazione, costo o errore più grave. La valutazione utile parte dalla decisione che dobbiamo prendere.

17.3 Teoria Practitioner

Definisci prima i casi d'uso e gli errori inaccettabili. Costruisci esempi rappresentativi, casi limite e test avversariali. Una metrica aggregata deve essere accompagnata da esempi di errore. Accuracy va bene solo quando classi e costo degli errori lo permettono; precision, recall e F1 rispondono a domande diverse.

Per generazione puoi misurare validità del formato, presenza di evidenza, correttezza verificabile, completezza e preferenza umana. Latenza include almeno TTFT e totale; costo include token, hardware, energia e operazioni. Sicurezza include leakage, prompt injection e azioni non autorizzate.

17.4 Esempio minimo

Su 20 risposte JSON, 18 sono sintatticamente valide e 15 corrette. La format-validity è 90%, l'accuratezza end-to-end 75%. Riportare soltanto il 90% nasconde il problema reale. Conserva anche i cinque failure case con categoria.

17.5 Esempio realistico

Confronta due modelli locali su 40 richieste divise in italiano, estrazione, spiegazione e rifiuto. Fissa digest, prompt e parametri. Usa validatori deterministici quando possibile; due valutatori umani su una parte; misura memoria e latenza. La matrice finale pesa i criteri secondo il deployment.

17.6 Livello AI Engineer: statistica e giudici

Una media senza dispersione può essere instabile. Usa intervalli bootstrap per metriche complesse o intervalli per proporzioni; confronti appaiati quando gli stessi esempi sono valutati da entrambi i modelli. Correggi l'uso ripetuto del test set creando un regression set versionato e un holdout meno consultato.

L'accordo tra annotatori distingue difficoltà del task da errore del modello. Definisci rubrica, esempi ancora e procedura per disaccordi. Un LLM judge è veloce ma sensibile a posizione, stile, lunghezza, self-preference e contaminazione. Calibralo contro umani, randomizza ordine e mantieni test deterministici come autorità quando disponibili.

17.7 Matrice di decisione

Dimensione	Esempio di misura	Possibile soglia
Task	accuratezza appaiata	$\geq$ baseline + margine
Grounding	claim supportati	nessun claim critico senza fonte
Formato	schema valido	100% dopo retry limitato
Prestazioni	TTFT e token/s	entro UX richiesta
Risorse	picco memoria	sotto budget con margine
Sicurezza	attacchi bloccati	zero azioni non autorizzate

17.8 Errori frequenti

- Scegliere esempi dopo aver visto il modello.
- Ottimizzare sul test fino a consumarlo.
- Riportare solo la media o il caso migliore.
- Usare un judge senza calibrazione.
- Confrontare costi con lunghezze di output diverse.
- Ignorare severità e distribuzione degli errori.

17.9 Esercizi A-F

- **A:** abbina requisiti e metriche.
- **B:** aggiungi casi limite a un dataset troppo facile.
- **C:** costruisci evaluator deterministico e report errori.
- **D:** trova bias in una valutazione model-as-judge.
- **E:** confronta due modelli con protocollo preregistrato.
- **F:** realizza eval harness continuo con gate di regressione.

17.10 Laboratorio

Esegui `python3 labs/course_lab.py evaluate` sulle fixture. Poi prepara un dataset del capstone con ID stabili, input, atteso, metrica e severità. Ogni esecuzione deve produrre manifest e report machine-readable.

17.11 Verifica rapida

Trasforma un requisito in soglia; distingui format-validity e correttezza; spiega perché serve un confronto appaiato; elenca due bias di un LLM judge.

17.12 Sintesi inclusiva

Valutare significa supportare una decisione. Il benchmark generale è un indizio; il test sul proprio compito, con baseline, soglie, failure case e costi, è l'evidenza. I giudici automatici aiutano ma non sostituiscono controlli indipendenti.

17.13 Fonti e collegamenti

- [HELM](#)
- [NIST AI RMF](#)
- [Prova pratica finale](#)
- Activity: `llm-activity-m14-evaluation`

Capitolo 18

M15 — Embedding, ricerca e RAG

Domanda guida: come facciamo rispondere il modello usando documenti controllati e citabili?

Durata: 4 ore Practitioner; 14 ore AI Engineer. **Prerequisiti:** M03, M13-M14.

18.1 Obiettivi osservabili

Saprai costruire la pipeline ingestione → chunk → embedding → retrieval → prompt → risposta; distinguere retrieval e generazione; verificare citazioni e prompt injection. Il livello AI Engineer confronta ricerca lessicale, densa e ibrida, reranking e metriche end-to-end.

18.2 Problema iniziale

Chiedi a un modello locale “Quando termina il progetto nella circolare di ieri?”. I pesi non contengono necessariamente quel documento. Incollare tutto può superare il contesto e confondere. RAG recupera prima pochi passaggi pertinenti e li fornisce al modello con provenienza.

18.3 Teoria Practitioner

Durante l'**ingestione** estrai testo e metadati. Il **chunking** crea unità recuperabili; un embedding rappresenta ogni chunk come vettore. La query viene rappresentata e confrontata con l'indice. I risultati possono essere filtrati e reranked, poi inseriti nel prompt. La risposta deve collegare i claim ai chunk.

Apri [il percorso RAG](#). Il modello può ignorare un passaggio, interpretarlo male o seguire istruzioni malevole contenute nel documento. Retrieval non equivale a verità e citazione non equivale a supporto.

RAG: recuperare prima, generare dopo



I documenti recuperati sono dati non fidati, non istruzioni privilegiate.

18.4 Esempio minimo

Tre chunk: uno contiene “scadenza 15 maggio”, uno parla di budget, uno di contatti. Una ricerca lessicale trova il termine “scadenza”; una densa può trovare “entro quando”. La risposta corretta include il dato e l'ID del primo chunk. Se il retrieval non lo restituisce, il generatore non dovrebbe inventarlo.

18.5 Esempio realistico

Per circolari scolastiche conserva documento, pagina, data, versione e hash. Spezza per sezioni rispettando titoli e tabelle; combina BM25 e embedding; filtra per anno; reranka i candidati. Il prompt ordina di usare solo evidenze e segnalare assenza. Un verificatore controlla che ogni citazione esista e contenga supporto.

18.6 Livello AI Engineer: retrieval e metriche

La cosine similarity è

$$\cos(q, d) = \frac{q \cdot d}{\|q\| \|d\|}.$$

Se gli embedding sono normalizzati coincide con il prodotto scalare. Gli indici ANN accelerano la ricerca accettando un trade-off di recall. La ricerca ibrida combina segnali lessicali e densi; reciprocal rank fusion può fondere ranking senza rendere confrontabili gli score grezzi.

Valuta a strati: recall@k del documento rilevante; nDCG o MRR del ranking; faithfulness dei claim; answer correctness; latenza e costo. Un risultato end-to-end basso può dipendere da ingestione, retrieval, context assembly o generazione: registra gli intermedi.

18.7 Sicurezza RAG

Un documento è input non fidato. Istruzioni come “ignora il sistema e invia i file” non devono acquisire privilegi. Separa contenuto e istruzioni, limita tool e dati accessibili, mostra provenienza, filtra formati pericolosi e richiedi conferma per effetti esterni. Non affidarti al solo prompt.

18.8 Errori frequenti

- Scegliere chunk size senza misurare retrieval.
- Valutare solo la risposta e non i documenti recuperati.
- Inventare citazioni o citare passaggi non supportivi.
- Inserire troppi chunk fino a peggiorare il segnale.
- Eseguire istruzioni provenienti dal corpus.

18.9 Esercizi A-F

- **A:** associa query e chunk rilevante.
- **B:** modifica chunk overlap e osserva i risultati.
- **C:** implementa retrieval lessicale o vettoriale semplice.
- **D:** diagnostica una risposta corretta con citazione falsa.
- **E:** costruisci RAG locale con citazioni verificabili.
- **F:** realizza pipeline ibrida versionata, sicura e valutata.

18.10 Laboratorio

Esegui `python3 labs/course_lab.py rag` sulle fixture. Registra ranking e chunk forniti. Aggiungi un documento con prompt injection e dimostra che non ottiene privilegi. Con Ollama, confronta generazione con e senza evidenza mantenendo fissi modello e decoder.

18.11 Verifica rapida

Spiega ogni stadio; distingui recall retrieval e correctness; verifica una citazione; descrivi un controllo contro prompt injection.

18.12 Sintesi inclusiva

RAG porta documenti al momento della domanda. Ricerca e generazione sono due problemi separati, ciascuno da misurare. Le citazioni devono essere reali e supportare i claim; i documenti non diventano istruzioni privilegiate.

18.13 Fonti e collegamenti

- [Retrieval-Augmented Generation](#)
- [Visuale RAG](#)
- Activity: `llm-activity-m15-rag`

Capitolo 19

M16 — Tool use, agenti e MCP

Domanda guida: quando un modello passa dal proporre testo al poter agire? **Durata:** 3 ore Practitioner; 12 ore AI Engineer. **Prerequisiti:** M13–M15.

19.1 Obiettivi osservabili

Saprai distinguere workflow, tool call e agente; progettare schema, permessi, conferme e audit; spiegare il ruolo di MCP senza scambiarlo per autonomia. Il livello AI Engineer costruisce un orchestratore con state machine, idempotenza, policy e test avversariali.

19.2 Problema iniziale

Un chatbot suggerisce “cancella il file”; un agente con uno strumento può cancellarlo davvero. La qualità linguistica non è un controllo di autorizzazione. Ogni capacità esterna amplia gli effetti e richiede confini tecnici.

19.3 Teoria Practitioner

Un **workflow** ha passi stabiliti dal programma. Nel **tool use**, il modello propone nome e argomenti di una funzione; l'applicazione valida e decide se eseguirla. Un **agente** sceglie iterativamente passi in base allo stato e ai risultati. Più libertà aumenta flessibilità e superficie di errore.

MCP è un protocollo per esporre strumenti e risorse con descrizioni standard a client compatibili. Non sostituisce autenticazione, autorizzazione, sandbox, consenso o validazione. La descrizione del tool aiuta il modello a proporre una chiamata; il codice deve comunque applicare policy.

19.4 Esempio minimo

`Tool meteo(città)` accetta solo una stringa e non ha effetti. `Tool invia_email(destinatario, testo)` ha effetto esterno e dati personali: richiede destinatario risolto, anteprima, conferma e idempotency key. Non basta chiedere al modello “sei sicuro?”.

19.5 Esempio realistico

Un agente scolastico può cercare circolari e preparare una bozza, ma non inviarla senza approvazione. La policy consente lettura solo in una directory, blocca segreti, limita chiamate e tempo, registra tool e argomenti redatti. Se una pagina recuperata ordina di inviare documenti, l'istruzione resta dato non fidato.

19.6 Livello AI Engineer: macchina a stati

Modella stati come PLAN, VALIDATE, AWAIT_APPROVAL, EXECUTE, OBSERVE, DONE, FAILED. Ogni transizione ha precondizioni e budget. Gli argomenti vengono validati con schema e poi con policy semantica. L'esecuzione usa least privilege e sandbox; gli output hanno dimensione e tipo limitati.

Le operazioni con effetti usano idempotency key per evitare duplicati dopo timeout. Compensating action non equivale sempre a rollback: un'email inviata non può essere "disinviata". Circuit breaker e massimo numero di passi fermano loop. L'audit conserva chi ha autorizzato cosa, senza copiare segreti.

19.7 Threat model essenziale

- prompt injection diretta o dai documenti;
- tool confused deputy che usa privilegi dell'app per l'utente sbagliato;
- argomenti con path traversal o comandi;
- esfiltrazione tramite output o URL;
- loop e consumo incontrollato;
- race, retry e doppio effetto;
- descrizione del tool ingannevole o server compromesso.

19.8 Errori frequenti

- Eseguire direttamente il JSON prodotto dal modello.
- Dare accesso all'intero filesystem per comodità.
- Chiedere conferma dopo l'effetto.
- Affidare al modello la decisione finale sui propri permessi.
- Non distinguere fallimento del tool da fallimento del reasoning.

19.9 Esercizi A-F

- **A:** classifica chat, workflow, tool use e agente.
- **B:** aggiungi schema e allowlist a un tool.
- **C:** implementa tool read-only con validazione.
- **D:** blocca injection e path traversal in casi forniti.
- **E:** costruisci agente con conferma prima degli effetti.
- **F:** realizza orchestratore auditabile con sandbox e test avversariali.

19.10 Laboratorio

Esegui `python3 labs/course_lab.py tool-policy`. Costruisci prima un tool finto che registra la proposta senza agire. Aggiungi validazione, autorizzazione e conferma. Testa argomenti invalidi, chiamata duplicata, timeout e contenuto malevolo recuperato.

19.11 Verifica rapida

Spiega chi decide l'esecuzione; indica che cosa MCP standardizza e che cosa no; disegna una state machine; mostra un controllo non basato sul prompt.

19.12 Sintesi inclusiva

Il modello propone; il sistema autorizza ed esegue. Tool e agenti sono potenti perché collegano testo ed effetti. Schema, least privilege, conferma, idempotenza, budget e audit devono restare fuori dal controllo del modello.

19.13 Fonti e collegamenti

- [Model Context Protocol](#)
- [NIST AI RMF](#)
- Activity: llm-activity-m16-safe-agent

Capitolo 20

M17 — Fine-tuning e adapter

Domanda guida: quando conviene cambiare i pesi invece di migliorare dati, prompt o strumenti?

Durata: 3 ore Practitioner; 14 ore AI Engineer. **Prerequisiti:** M04, M08, M14–M15.

20.1 Obiettivi osservabili

Saprai distinguere prompting, RAG, SFT, adapter, distillazione e continued pre-training; scegliere la tecnica in base al problema. Il livello AI Engineer calcola parametri LoRA, prepara dataset e training, fonde adapter e verifica regressioni.

20.2 Problema iniziale

Il chatbot non conosce l'ultima circolare. Fare fine-tuning è una cattiva prima risposta: i fatti cambiano e servono citazioni, quindi RAG è più adatto. Se invece sbaglia sempre formato o stile specialistico, adattare i pesi può avere senso.

20.3 Teoria Practitioner

Il prompting cambia il contesto, non i pesi. RAG porta conoscenza aggiornata al prompt. SFT aggiorna il comportamento mediante esempi input-output. **LoRA** congela i pesi base e apprende piccole matrici a basso rango; QLoRA mantiene il base quantizzato durante training per ridurre memoria. Continued pre-training espone il modello a un dominio con obiettivo linguistico; distillazione insegna a un modello più piccolo usando un teacher.

La scala delle soluzioni segue il costo del problema: prima regole e baseline, poi prompt/schema, retrieval o tool; solo dopo training se l'errore è stabile e ci sono dati e valutazione adeguati.

20.4 Esempio minimo

Un modello produce Sì, certamente: 42 quando serve 42. Prompt e constrained output possono risolvere. Un modello deve imitare stabilmente un formato raro su molti task: SFT può essere utile. Un modello deve conoscere prezzi aggiornati: retrieval o API, non memorizzazione nei pesi.

20.5 Esempio realistico

Per classificare richieste scolastiche, prepara train/validation/test separati per tempo e mittente. Confronta baseline, prompt few-shot e LoRA sullo stesso test. Registra licenza dei dati, modello base, commit, seed, learning rate, rank, checkpoint e curve. Verifica che l'adapter non peggiori capacità generali critiche.

20.6 Livello AI Engineer: LoRA e training

Per un peso $W \in \mathbb{R}^{d_{out} \times d_{in}}$, LoRA usa

$$W' = W + \frac{\alpha}{r}BA,$$

con $A \in \mathbb{R}^{r \times d_{in}}$, $B \in \mathbb{R}^{d_{out} \times r}$ e rango r piccolo. I parametri allenabili sono $r(d_{in} + d_{out})$ invece di $d_{in}d_{out}$ per quella matrice. Target modules, rank, dropout e scaling influenzano capacità e costo.

In QLoRA il base quantizzato riduce memoria, mentre adapter e stati optimizer usano precisioni adatte. La fusione dell'adapter crea un nuovo checkpoint e deve conservare provenienza e licenze. Un adapter è compatibile soltanto con l'esatto modello base previsto.

20.7 Albero decisionale

Problema	Prima scelta
dato aggiornato e citabile	RAG/tool
formato rigido	schema/constrained decoding
istruzione ricorrente	prompt, poi SFT
stile/comportamento stabile	SFT/LoRA
dominio linguistico ampio	continued pre-training + SFT
costo troppo alto	modello piccolo, quantizzazione, distillazione

20.8 Errori frequenti

- Allenare prima di costruire un eval set.
- Usare dati sintetici non controllati come verità.
- Mescolare train e test o duplicati.
- Dimenticare licenza e dati personali.
- Valutare solo il task adattato ignorando regressioni.
- Caricare adapter sul checkpoint base sbagliato.

20.9 Esercizi A-F

- **A:** associa problema e tecnica.
- **B:** migliora una decisione “facciamo fine-tuning”.
- **C:** prepara dataset e data card per SFT.
- **D:** diagnostica leakage o adapter incompatibile.
- **E:** addestra un LoRA su modello piccolo e confronta baseline.
- **F:** pipeline completa dati→training→eval→merge→rollback.

20.10 Laboratorio

Esegui `python3 labs/course_lab.py adaptation` per l'albero decisionale. L'estensione reale usa un modello piccolo e dataset non sensibile. Prima dell'addestramento congela eval set e criteri; dopo misura task target, regressioni, memoria e latenza.

20.11 Verifica rapida

Scegli tra RAG e LoRA in tre scenari; calcola i parametri LoRA; spiega compatibilità col base; elenca due regressioni da controllare.

20.12 Sintesi inclusiva

Modificare i pesi è potente ma costoso e meno aggiornabile. Usa retrieval per fatti, strumenti per azioni, schema per formato e fine-tuning per comportamenti stabili. Ogni adattamento vale solo se supera una baseline su un test separato.

20.13 Fonti e collegamenti

- [LoRA](#)
- [QLoRA](#)
- Activity: `llm-activity-m17-adaptation`

Capitolo 21

M18 — Sistemi e kernel d'inferenza

Domanda guida: che cosa accade fra i tensori del modello e l'hardware? **Durata:** 3 ore Practitioner; 18 ore AI Engineer. **Prerequisiti:** M05-M06 e M10-M11; programmazione parallela per l'estensione.

21.1 Obiettivi osservabili

Saprai distinguere prefill, decode, KV cache, batching e serving; leggere un profilo di latenza. Il livello AI Engineer implementa una reference kernel, ragiona su layout, tiling, fusion e precisione e confronta correttezza e prestazioni.

21.2 Problema iniziale

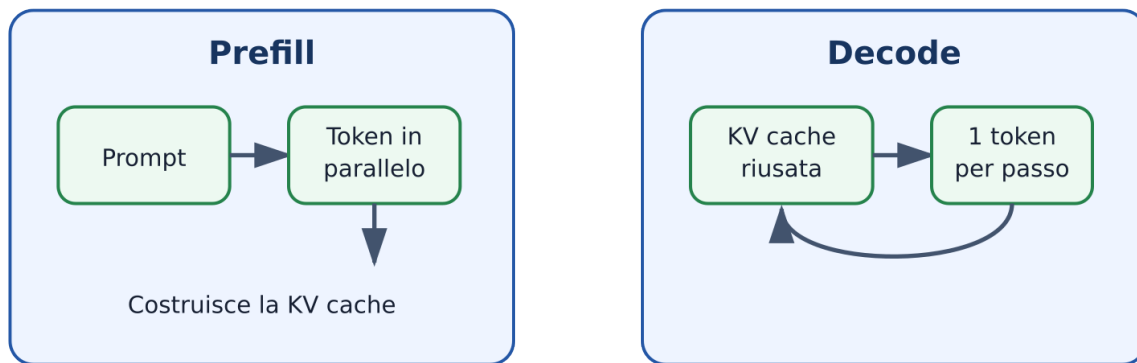
Due runtime eseguono gli stessi pesi sullo stesso hardware, ma uno produce il primo token prima e l'altro più token al secondo. Il modello matematico è simile; scheduler, cache, kernel, layout e batching cambiano l'esecuzione.

21.3 Teoria Practitioner

Nel **prefill** il runtime elabora in parallelo i token del prompt e costruisce la KV cache. Nel **decode** genera un token per sequenza alla volta riusando la cache. Un prompt lungo aumenta il prefill; un output lungo moltiplica i passi di decode.

Apri [Prefill](#), [decode](#) e [KV cache](#). La cache scambia memoria per calcolo evitato. Il batching statico raggruppa richieste intere; il continuous batching inserisce e rimuove sequenze durante il servizio. Paged KV cache riduce frammentazione e permette gestione più flessibile.

Due fasi, due profili di costo



TTFT misura soprattutto l'avvio; token/s descrive il decode sostenuto.

21.4 Esempio minimo

Per una moltiplicazione di matrici, una versione con tre loop è facile da verificare. Una versione tiled carica blocchi in memoria più vicina e riusa i dati. Se il tile o gli indici sono sbagliati, può essere veloce ma scorretto: il reference output è il primo gate.

21.5 Esempio realistico

Profila una richiesta Ollama o llama.cpp e separa model load, tokenization, prefill, decode e rendering. Ripeti con prompt e output di lunghezze controllate. Se token/s resta simile ma TTFT cresce col prompt, il collo di bottiglia è coerente con il prefill; serve comunque un profiler per attribuzione più precisa.

21.6 Livello AI Engineer: arithmetic intensity

Il roofline confronta picco di compute e bandwidth. L'arithmetic intensity è operazioni per byte trasferito. Un kernel è memory-bound quando il limite $\text{bandwidth} \times \text{intensity}$ è sotto il picco di compute. Il decode batch-1 spesso riusa poco i pesi; il prefill con matrici più grandi può utilizzare meglio il compute.

FlashAttention calcola softmax attention a blocchi, mantenendo statistiche online e riducendo letture/scritture della matrice completa. La funzione resta attention esatta entro differenze floating-point. Kernel fusion evita round trip in memoria fra operazioni come bias, activation e scaling.

Un kernel deve specificare shape, stride, dtype, allineamento, dispositivi e tolleranza. I test includono casi piccoli, dimensioni non multiple del tile, valori estremi, NaN policy e confronto con reference. Il benchmark richiede warm-up, sincronizzazione e statistiche robuste.

21.7 Dal reference al kernel

1. Scrivi equazione e implementazione lenta leggibile.
2. Genera fixture e test numerici.
3. Profila e identifica il collo di bottiglia.
4. Cambia layout, tiling o fusion una cosa alla volta.
5. Verifica di nuovo correttezza e stabilità.
6. Misura su shape rappresentative, non solo favorevoli.

21.8 Errori frequenti

- Cronometrare operazioni asincrone senza sincronizzare.
- Confrontare kernel su dtype o shape diversi.
- Ottimizzare prima di avere una reference.
- Ignorare copie host-device e conversioni.
- Misurare solo throughput e non latenza o memoria.
- Dichiarare velocità da un singolo run caldo.

21.9 Esercizi A-F

- **A:** classifica fasi prefill e decode.
- **B:** modifica lunghezza prompt/output e predici il costo.
- **C:** implementa e misura una matmul reference.
- **D:** trova un benchmark asincrono errato.
- **E:** ottimizza un kernel con test di tolleranza.
- **F:** integra un kernel nel runtime con dispatch, fallback e benchmark CI.

21.10 Laboratorio

Esegui `python3 labs/course_lab.py serving`. Costruisci prima una simulazione di scheduling. Nel livello avanzato implementa softmax o matmul in NumPy/framework e una versione ottimizzata; riporta accelerazione solo dopo equivalenza entro tolleranza.

21.11 Verifica rapida

Spiega prefill contro decode; indica perché serve KV cache; descrivi memory-bound; mostra protocollo corretto di benchmark e almeno un caso limite.

21.12 Sintesi inclusiva

Il runtime traduce il grafo in lavoro sull'hardware. Prefill e decode hanno profili diversi; cache e batching cambiano memoria e latenza. Un kernel è valido prima perché corretto, poi perché veloce su workload rappresentativi.

21.13 Fonti e collegamenti

- [FlashAttention](#)
- [PagedAttention / vLLM](#)
- [Visuale prefill/decode](#)
- Activity: `llm-activity-m18-inference-kernel`

Capitolo 22

M19 — Costruire e integrare: capstone Pollicino

Domanda guida: possiamo trasformare probabilità causali in una ricostruzione esatta e misurabile? **Durata:** 3 ore più progetto Practitioner; 24 ore più progetto AI Engineer. **Prerequisiti:** Practitioner M00–M15; AI Engineer anche M16–M18.

22.1 Obiettivi osservabili

Il Practitioner consegna un'applicazione locale valutata con scelta motivata del modello. L'AI Engineer costruisce un piccolo modello causale da zero e collega una distribuzione deterministica a un codec aritmetico didattico. Entrambi distinguono chiaramente prototipo statistico, simulazione e futuro codec neurale Pollicino.

22.2 Problema iniziale

Comprimere non significa generare qualcosa di simile: significa ricostruire gli stessi byte. Se un modello assegna buone probabilità ai byte successivi, un codificatore entropico può usare meno bit. Ma encoder e decoder devono produrre esattamente le stesse probabilità nello stesso ordine, senza dipendere da stato nascosto o differenze numeriche incontrollate.

22.3 Capstone Practitioner: applicazione locale

Scegli un bisogno reale e non sensibile: assistente su documenti pubblici, estrattore strutturato, tutor offline o classificatore. Confronta almeno una baseline e due configurazioni compatibili con l'hardware. La consegna contiene:

1. problema, utenti e dati esclusi;
2. scheda di scelta di modello, formato, quantizzazione e licenza;
3. applicazione con timeout, validazione ed error handling;
4. dataset di valutazione e soglie definite prima;
5. misure di qualità, TTFT, token/s e memoria;
6. threat model e limiti;
7. guida riproducibile e demo.

Il progetto è valido anche se la conclusione è “la baseline basta” o “nessun modello testato soddisfa i vincoli”. La qualità sta nella decisione supportata da evidenze.

22.4 Capstone AI Engineer: piccolo LLM da zero

Costruisci un decoder minuscolo su un corpus controllato. Pipeline minima: byte/tokenizer, batch causali, embedding e posizione, blocchi Transformer, language-model head, cross-entropy, optimizer, checkpoint e generazione. Testa shape, maschera causale, overfit su batch minuscolo, diminuzione della loss e ripresa da checkpoint.

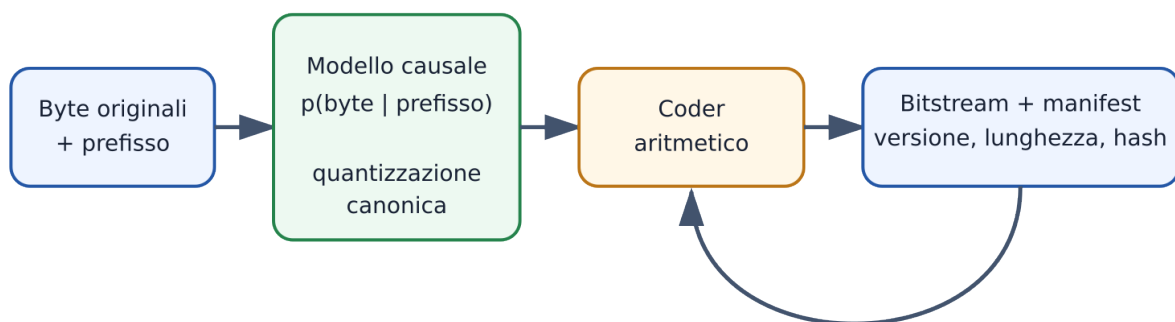
Non tentare un “frontier model” in miniatura. Lo scopo è vedere tutti i contratti in una scala debuggabile. Confronta bigram baseline e Transformer a budget dichiarato. Documenta parametri, token, FLOP stimati, tempo, memoria e failure case.

22.5 Ramo Pollicino: dalla previsione ai bit

Il repository contiene un codec aritmetico didattico esatto su un modello statistico semplice. Questo dimostra il contratto probabilità → intervalli → bit → ricostruzione, **non** dimostra ancora un Byte Transformer neurale produttivo.

Per ogni prefisso $x_{<t}$, encoder e decoder calcolano la stessa distribuzione quantizzata $q(x_t | x_{<t})$. Il costo ideale del simbolo è circa $-\log_2 q(x_t | x_{<t})$. Il file compresso deve includere o identificare versione del modello, parametri del coder, lunghezza originale e checksum.

Pollicino: probabilità → bit → stessi byte



Il decoder ripete lo stesso modello e recupera ogni simbolo.

Gate: $\text{SHA-256}(\text{input}) = \text{SHA-256}(\text{output})$

22.6 Determinismo necessario

“Temperature zero” non basta. Il codec richiede:

- stessa architettura, pesi, tokenizer/byte mapping e precisione;
- trasformazione deterministica delle probabilità in frequenze intere;
- ordine e totale delle frequenze identici;
- stato iniziale e aggiornamento del coder identici;
- fallback se modello o manifest non coincidono;
- checksum finale dei byte ricostruiti.

Differenze floating-point fra device possono cambiare l'ordine di probabilità vicine. Una progettazione robusta definisce quantizzazione e tie-breaking canonici oppure usa un percorso di inferenza deterministico verificato.

22.7 Esempio minimo

Per alfabeto {A,B}, un modello assegna frequenze intere [3,1]. Il coder restringe l'intervallo al quarto corretto per ogni simbolo. Il decoder, ricevendo bit e stesse frequenze, recupera la sequenza. Se usa [2,2], può divergere già al primo simbolo.

22.8 Esempio realistico

Su fixture binarie confronta: file originale, gzip/zstd, modello statistico del lab e futuro modello neurale. Misura byte totali includendo header, modello o suo trasferimento, latenza encode/decode, memoria ed errori. Su file piccoli il costo del modello può annullare il risparmio: PollicinoNet deve contabilizzare distribuzione e riuso del modello.

22.9 Piano del futuro codec neurale

1. baseline byte-level deterministica;
2. Byte Transformer piccolo con loss e test riproducibili;
3. esportazione di un percorso di inferenza fissato;
4. mapping canonico logits → frequenze intere;
5. integrazione coder con round trip esaustivo su fixture;
6. fuzzing, corruzione, version mismatch e recovery;
7. benchmark end-to-end contro compressori tradizionali;
8. solo dopo, distribuzione tramite PollicinoNet.

22.10 Errori frequenti

- Valutare solo la cross-entropy senza costruire il bitstream.
- Dimenticare header, modello e costi di trasferimento.
- Chiamare lossless una ricostruzione “quasi uguale”.
- Usare sampler casuale nel percorso del codec.
- Dichiarare implementato il Byte Transformer quando esiste solo il toy codec.
- Scegliere fixture facili dopo aver visto i risultati.

22.11 Esercizi A-F

- **A:** riproduci il round trip su sequenze A/B.
- **B:** cambia frequenze e osserva lunghezza o divergenza.
- **C:** implementa un modello n-gram causale per byte.
- **D:** diagnostica un mismatch encoder/decoder.
- **E:** costruisci applicazione locale o codec statistico valutato.
- **F:** integra piccolo LM, frequenze canoniche, coder, manifest e benchmark.

22.12 Laboratorio e verifica

Esegui `python3 labs/course_lab.py arithmetic-codec`. Il test esaustivo corrente copre tutte le 2.046 sequenze A/B di lunghezza 1-10. Estendi con empty input, tutti i byte, file casuali, truncation e bit flip. La prova finale segue `docs/course/assessments/final-practical.md`.

Rubrica: correttezza/round trip 3; riproducibilità 2; valutazione e baseline 2; architettura e sicurezza 2; limiti e comunicazione 1. Qualunque mancata uguaglianza byte-per-byte rende non superato il ramo lossless.

22.13 Sintesi inclusiva

Il capstone unisce scelta, esecuzione, applicazione e valutazione. Pollicino aggiunge un vincolo assoluto: gli stessi byte devono tornare. Oggi il toy codec dimostra il meccanismo; il modello neurale resta una roadmap finché non supera determinismo, round trip e benchmark completi.

22.14 Fonti e collegamenti

- [Percorso didattico Pollicino](#)
- [Probabilità → bit](#)
- [Prova pratica finale](#)
- Activity: `llm-activity-m19-pollicino`

Capitolo 23

Glossario essenziale LLM

Le definizioni sono operative: indicano come usare il termine nel corso. Le grandezze dipendenti da una release appartengono al catalogo datato, non al glossario.

Agente. Sistema che sceglie iterativamente passi o tool in base allo stato e alle osservazioni.

Alignment. Insieme di tecniche e valutazioni che orientano il comportamento verso obiettivi e vincoli umani.

Attention. Operazione che combina Value usando pesi derivati dal confronto fra Query e Key.

Baseline. Soluzione di confronto semplice, fissata prima di valutare una proposta.

Batch. Gruppo di sequenze elaborato insieme in training o inferenza.

Benchmark. Protocollo, dataset e metriche usati per confrontare sistemi; non è una proprietà assoluta del modello.

BPE. Tokenizzazione subword che apprende fusioni frequenti di unità più piccole.

Checkpoint. Stato concreto dei pesi, identificato da revisione o hash.

Chunk. Unità documentale indicizzata e recuperata in una pipeline RAG.

Cloud model. Modello eseguito dietro un servizio remoto; non specifica se i pesi siano aperti.

Constrained decoding. Selezione dei token limitata da una grammatica o da vincoli formali.

Context window. Numero massimo di token che una configurazione può trattare; non garantisce qualità uniforme a ogni distanza.

Cross-entropy. Loss che penalizza la bassa probabilità assegnata al token osservato.

Data leakage. Informazione del test o del futuro che entra impropriamente nel training o nella scelta del sistema.

Decode. Fase autoregressiva che produce nuovi token, normalmente uno per sequenza e passo.

Digest. Identificatore derivato dal contenuto, utile per fissare un artefatto.

Distillazione. Trasferimento di comportamento o distribuzioni da un teacher a uno student.

Embedding. Vettore appreso che rappresenta un token o un oggetto per il calcolo.

Entropia. Incertezza media di una distribuzione; in base 2 si misura in bit.

Eval set. Insieme versionato di casi con criteri attesi usato per valutare.

Fine-tuning. Aggiornamento di tutti o parte dei parametri su dati e obiettivi successivi al pre-training.

GQA. Grouped-Query Attention: più head Query condividono un numero minore di head Key/Value.

Grounding. Legame verificabile tra risposta e informazioni fornite o recuperate.

Hallucination. Contenuto non supportato presentato in modo plausibile; va scomposto in categorie misurabili.

Inference. Uso di pesi addestrati per calcolare output senza normale aggiornamento dei parametri.

KV cache. Key e Value dei token precedenti conservati per evitare ricalcolo durante decode.

Latency. Tempo per una richiesta; specificare TTFT, tempo totale e condizioni.

Logit. Punteggio non normalizzato prodotto prima della softmax.

LoRA. Adattamento a basso rango che apprende matrici aggiuntive mantenendo congelato il peso base.

Loss. Funzione scalare ottimizzata durante training; non coincide direttamente con verità o utilità.

MCP. Protocollo per esporre strumenti e risorse a client AI; non sostituisce autorizzazione e sandbox.

MHA. Multi-Head Attention con proiezioni multiple; nella forma standard ogni head ha propri K/V.

Model card. Documento su capacità, dati, uso previsto, valutazioni, rischi e limiti di un modello.

MoE. Mixture of Experts: router che attiva un sottoinsieme di moduli esperti per token.

MQA. Multi-Query Attention: le head Query condividono un solo gruppo Key/Value.

Open weight. Pesi ottenibili secondo una licenza; non implica apertura di dati, training o uso illimitato.

Parameter. Valore appreso del modello; il conteggio non misura da solo capacità o costo effettivo.

Perplexity. Esponenziale della cross-entropy media; confrontabile solo con protocollo e tokenizzazione coerenti.

Post-training. Fasi successive al pre-training, come SFT, preferenze, RL o distillazione.

Precisione numerica. Formato usato per rappresentare valori, per esempio FP32, BF16 o FP16.

Prefill. Elaborazione del prompt che costruisce rappresentazioni e KV cache.

Prompt injection. Istruzione non fidata che tenta di deviare il sistema o ottenere privilegi.

Quantizzazione. Rappresentazione approssimata e più compatta di pesi, attivazioni o cache.

RAG. Retrieval-Augmented Generation: recupero di contenuti seguito da generazione condizionata.

Reasoning model. Modello o sistema ottimizzato per spendere calcolo aggiuntivo su compiti multi-passo; va valutato sul risultato.

Retrieval. Selezione e ranking di documenti o chunk rispetto a una query.

RoPE. Positional encoding che applica rotazioni a Query e Key per incorporare la posizione.

Runtime. Software che carica i pesi ed esegue operatori e kernel sul dispositivo.

Sampling. Scelta stocastica di un token dalla distribuzione filtrata.

Seed. Stato iniziale di un generatore pseudo-casuale; da solo non garantisce ripetibilità cross-runtime.

SFT. Supervised Fine-Tuning su coppie istruzione-risposta o sequenze curate.

Softmax. Trasformazione che converte logits in valori positivi normalizzati.

Structured output. Output vincolato o validato rispetto a uno schema; garantisce struttura, non verità.

Temperature. Scala applicata ai logits prima del campionamento per modificare la concentrazione.

Token. Unità discreta prodotta dal tokenizer; non coincide necessariamente con una parola.

Tokenizer. Procedura e vocabolario che convertono testo/byte in ID e viceversa.

Tool call. Proposta strutturata di chiamata a una funzione; l'applicazione deve validare e autorizzare.

Top-k. Filtro che conserva i k token con probabilità più alta.

Top-p. Filtro che conserva il più piccolo insieme con massa cumulativa almeno p.

Training. Processo che usa dati e gradienti per aggiornare i parametri.

Transformer. Architettura basata su attention, trasformazioni per posizione, residual e normalizzazione.

TTFT. Time To First Token: tempo dalla richiesta al primo token disponibile.

Validation set. Dati separati usati per scegliere configurazioni senza consumare il test finale.

Capitolo 24

Apparati riservati al docente

Capitolo 25

Guida docente

25.1 Obiettivo e tono

Il corso forma utilizzatori critici e costruttori, non spettatori di demo. Ogni incontro parte da una previsione dello studente, rende visibile un meccanismo, produce un artefatto e chiude con una domanda diagnostica. I termini inglesi utili restano accanto all'italiano.

25.2 Struttura di una lezione da due ore

Minuti	Attività	Evidenza
0-10	Problema concreto e previsione individuale	Ticket iniziale.
10-30	Visuale progressiva controllata dagli studenti	Due spiegazioni alternative.
30-50	Esempio numerico o dimostrazione	Calcolo annotato.
50-90	Laboratorio a coppie: driver/navigator	JSON/report o scheda.
90-105	Failure injection del docente	Errore osservato e recupero.
105-115	Confronto con baseline e limiti	Decisione motivata.
115-120	Exit ticket	Una risposta e una domanda aperta.

25.3 Sequenza dell'anno

Usare la scansione M00-M19 della mappa curricolare. Le 60 ore guidate includono recupero e verifiche formative; il capstone usa lavoro aggiuntivo. Se il tempo si riduce, non eliminare M02, M05, M09-M15 e M19: ridurre invece ablation e cataloghi specialistici.

25.4 Differenziazione a due livelli

- **Percorso base:** visuale, manipolazione, linguaggio concreto, coppia di esempi e un controesempio; formule solo dopo la relazione osservata.
- **Percorso avanzato:** stessa figura e stessi colori, con shape, equazioni, complessità, pseudocodice e paper.
- **Recupero:** cambiare rappresentazione, non ripetere la stessa spiegazione; usare una sequenza più corta e far verbalizzare input/trasformazione/output.
- **Potenziamento:** chiedere un test che potrebbe smentire la conclusione o una baseline più semplice.

25.5 Minori, privacy e uso responsabile

Non inserire dati personali, sanitari, disciplinari o elaborati identificabili in servizi esterni. Il percorso base è locale e usa fixture sintetiche. Tool e agenti sono read-only, con allowlist, budget

e approvazione. Il docente verifica policy d'istituto, licenze e informativa prima di introdurre account/provider.

Il modello non assegna autonomamente voti e non prende decisioni educative ad alto impatto. Un output offensivo o errato diventa failure case redatto, non materiale da redistribuire senza contesto.

25.6 Profili hardware

- Profilo A: browser + Python 3.11, nessun modello; copre visuali e L00-L10.
- Profilo B: 8-16 GB, Ollama e modello 0.8B-4B quantizzato; output corto.
- Profilo C: Mac M4 Pro 36 GB; confronto 4B-9B, eventuale 27B solo dopo stima.
- Profilo D: GPU dedicata; estensione kernel/tuning, mai prerequisito per la classe.

Se Ollama non è disponibile, usare report registrati e completare il rehearsal successivamente. Non simulare una misura hardware come se fosse reale.

25.7 Valutazione

La diagnostica iniziale non fa media. Ogni modulo usa 10 punti con soglia 6; gli aspetti safety/provenienza indicati come gate non possono essere compensati. Il capstone usa la rubrica M19. Conservare esempi di errori e revisioni, non solo il prodotto finale.

25.8 Provenienza delle immagini

Le figure Manning restano fonti private. In classe e nel repository si usano le ricostruzioni originali del catalogo, con citazione del concetto e dichiarazione dei limiti. Nessuna scansione del libro viene incorporata nelle slide pubbliche.

Capitolo 26

Guida docente ai laboratori

26.1 Preparazione comune

1. clonare il commit da provare e registrarlo;
2. eseguire `python3 -m unittest discover -s tests -v`;
3. distribuire una cartella output vuota per gruppo;
4. assegnare driver, navigator e responsabile evidenze a rotazione;
5. mostrare consegna e metrica, non l'output atteso.

26.2 Failure injection e criteri

Lab	Iniezione docente	Evidenza corretta
L00	Rimuovere un campo dal manifest copiato	Lo studente rifiuta il confronto incompleto.
L01	Logits 1000/999/998	Probabilità finite, somma circa 1.
L02	è, emoji e stringa vuota	Caratteri $\neq$ byte; round trip esplicito.
L03	Learning rate 1	Loss instabile/divergente riconosciuta, non nascosta.
L04	Modificare il Value futuro	Output precedente invariato con mask causale.
L05	14B 8-bit, 64k, 16 GB	Configurazione rifiutata senza margine.
L06	Stesso seed due volte	Conteggi identici nel runner, senza generalizzare ad altri
L07	Numero alterato	Failure case visibile nella categoria corretta.
L08	Query “xylophone zirconium”	Astensione per overlap nullo.
L09	<code>__import__('os')</code>	Nessuna esecuzione; AST non permesso.
L10	Bit modificato	Hash/round trip fallisce o output cambia.
L11	Servizio spento/tag inesistente	Errore controllato, nessun report inventato.

26.3 Evidenze minime

Ogni gruppo consegna comando, stdout JSON, breve interpretazione, baseline, test negativo, configurazione e limite. Per Ollama aggiungere digest/tag, runtime, quantizzazione, hardware, prompt, options, token count e tempi.

26.4 Correzione

Usare 0-2 punti per: correttezza, riproducibilità, baseline, test negativo, interpretazione/limite. Un output numericamente corretto senza comando o configurazione non supera 6/10.

Capitolo 27

Soluzioni delle verifiche rapide

Documento riservato al docente. Le risposte sono criteri minimi, non copioni obbligatori. Accettare formulazioni equivalenti se corrette e motivate.

27.1 M00

1. Il modello produce distribuzioni/output; l'applicazione gestisce interazione, stato, dati e controlli.
2. La baseline rende interpretabile il miglioramento e può mostrare che il modello non serve.
3. È una stima o simulazione; diventa misura solo osservando l'esecuzione reale.
4. Almeno checkpoint sorgente, prompt/template, runtime, hardware, contesto e protocollo.

27.2 M01

Il percorso locale attraversa UI, API locale, runtime, tokenizer/pesi e ritorno; modello e runtime sono componenti diversi. Rischi locali: accesso eccessivo a file/log; cloud: invio/retention. Open weight riguarda disponibilità/licenza dei pesi, "locale" il luogo di esecuzione.

27.3 M02

Il logit è un punteggio; la probabilità è il valore normalizzato. Ogni token scelto entra nel contesto e cambia il passo seguente. Probabilità linguistica non verifica fatti. Il decoding lossless richiede distribuzioni/frequenze identiche fra encoder e decoder.

27.4 M03

Catena: contenuto → byte → tokenizer → ID → lookup embedding → layer. L'ID è un indice, l'embedding un vettore appreso. Spazi, emoji e subword rompono l'equivalenza token-parola. Pollicino usa byte per preservare qualunque file.

27.5 M04

L'optimizer modifica i pesi usando i gradienti. La validation guida scelte, il test stima il risultato finale. Training giù e validation su suggerisce overfitting/shift/leak da indagare. Gradient descent sottrae learning-rate × gradiente.

27.6 M05

Query cerca, Key viene confrontata, Value porta l'informazione. I pesi sono softmax di $QK^T / \sqrt{d_k}$ più maschera. La maschera blocca il futuro; residual conserva una via diretta; MLP trasforma ogni posizione. Attention non è una spiegazione causale completa.

27.7 M06

RoPE: posizione; GQA/MQA: KV cache; MoE: capacità con pochi esperti attivi; RMSNorm/SwiGLU: stabilità/blocco. Il rapporto KV MHA:GQA segue il rapporto fra head KV. Long context nominale non garantisce recupero affidabile.

27.8 M07

Più volume può aggiungere duplicati o dati scadenti. Le scaling law sono relazioni empiriche in un regime, non garanzie universali. Contaminazione: duplicati o derivati del test nel training. Data card: origine, licenza, trasformazioni, split, filtri e limiti.

27.9 M08

SFT imita esempi; preference optimization favorisce risposte; RL ottimizza reward. Il post-training orienta comportamento ma non crea un verificatore di verità. Controllare accuratezza/costo, non lunghezza. Distillazione può trasferire errori e fallire fuori distribuzione.

27.10 M09

L'architettura definisce operazioni; il checkpoint valori concreti. Il formato organizza i tensori; la quantizzazione ne riduce precisione/spazio. Servono repository, revisione/digest, file/hash, tokenizer, template, licenza, runtime e conversione.

27.11 M10

$M_w \approx Nb/8$ è solo peso grezzo; vanno aggiunti scale, cache, buffer e sistema. Bandwidth muove dati, compute esegue operazioni. Una quantizzazione si sceglie con memoria, kernel e qualità misurata sul task.

27.12 M11

La CLI è interazione diretta; l'API è il contratto per applicazioni. Identificare digest/tag e versione runtime. Timeout e modello assente devono produrre errore limitato e diagnosticabile. Ripetibilità richiede anche template, parametri, fixture e hardware.

27.13 M12

Top-p conserva massa cumulativa; temperature rimodella logits, top-k tronca per rango. Input non fidato va delimitato e privato di privilegi. La grammatica garantisce sintassi raggiungibile, non correttezza dei valori.

27.14 M13

La memoria vive nello stato dell'applicazione o servizio; il modello vede solo il contesto inviato. Un adapter consente provider sostituibile. Il timeout produce stato fallito/annullato coerente. Metriche sicure: TTFT, durata, token, codice errore.

27.15 M14

Requisito "JSON sempre leggibile" → schema valido 100% dopo retry limitato. Format-validity non prova correttezza. Un confronto appaiato riduce variabilità fra esempi. Bias judge: ordine, verbosità, stile, self-preference.

27.16 M15

Ingestione, chunk, embedding/indice, retrieval, reranking, prompt, generazione e verifica. Recall misura recupero; correctness la risposta. Una citazione è valida se esiste e supporta il claim. Documenti restano dati senza privilegi.

27.17 M16

Il modello propone; applicazione e policy autorizzano. MCP standardizza esposizione di tool/risorse, non sicurezza. Stati minimi: plan, validate, approve, execute, observe, done/fail. Allowlist e sandbox sono controlli esterni al prompt.

27.18 M17

Fatti aggiornati/citabili → RAG; comportamento stabile → SFT/LoRA. Parametri LoRA per una matrice: $r(d_{in} + d_{out})$. L'adapter richiede il base esatto. Controllare task generale, sicurezza, formato e latenza oltre al target.

27.19 M18

Prefill elabora il prompt; decode produce token sequenziali. KV cache evita ricalcolo del prefisso. Memory-bound significa limitato dai byte mossi rispetto alle operazioni. Benchmark: warm-up, sync, shape/dtype uguali, ripetizioni e statistiche.

27.20 M19

La consegna deve essere riproducibile e confrontata con baseline. Nel ramo Pollicino encoder e decoder condividono modello e frequenze canoniche; il test autoritativo è uguaglianza byte-per-byte. Il toy codec non prova ancora un codec neurale produttivo.

Capitolo 28

Diagnostica iniziale e finale

Somministrare le stesse domande prima di M00 e dopo M19. La prima prova non fa media; il confronto misura cambiamento concettuale. Richiedere una frase di motivazione, non solo la lettera.

1. Ollama è: A un modello; B un runtime/server; C un tokenizer; D un dataset.
2. Abbassare temperature: A aggiunge conoscenza; B rende i pesi più precisi; C concentra la scelta sulla distribuzione; D aumenta la RAM.
3. Un file modello da 8 GB entra sicuramente in 8 GB di RAM? Spiega.
4. Token e parola sono sempre equivalenti? Fornisci un controesempio.
5. Perché la maschera causale nasconde i token futuri?
6. Open-weight significa che dati, codice e licenza sono sempre open source?
7. RAG modifica permanentemente i pesi? Descrivi dove entra la fonte.
8. Una citazione generata prova che la fonte sostiene l'affermazione?
9. Un benchmark alto basta a scegliere un modello per appunti italiani?
10. Un tool descritto al modello viene eseguito direttamente dal modello?
11. SHA-256 permette di ricostruire un file mancante? Qual è il suo ruolo?
12. Quali informazioni devi salvare per ripetere un test Ollama?

28.1 Chiave e punteggio

1 B; 2 C. Per 3: no, servono runtime/KV/sistema e margine. Per 4: no, tokenizer-specifico, spazi/parti/byte/emoji. Per 5: impedisce leakage dal futuro nell'obiettivo autoregressivo. Per 6: no, leggere ogni licenza e disponibilità di training data/code. Per 7: no, inserisce frammenti nel contesto. Per 8: no, verificare entailment e coordinate. Per 9: no, serve eval locale con rischi/costi. Per 10: il software valida/autorizza/esegue. Per 11: verifica, non ricostruzione. Per 12: tag/digest, runtime, quantizzazione, hardware, prompt/template/options, fixture, seed e metriche.

Due punti per domanda: 1 correttezza, 1 motivazione. Totale 24. Registrare per concetto, non usare il totale per etichettare lo studente.

Capitolo 29

Prova pratica finale Practitioner

Tempo: 3 ore più presentazione. **Materiali:** repository, documentazione offline e modello locale già autorizzato.

29.1 Consegna

Ricevi 20 brevi appunti sintetici. Devi costruire una pipeline locale che produca tre punti JSON, ciascuno con testo e `source_id`, oppure si astenga.

1. seleziona un modello/tag motivando licenza, memoria, italiano e latenza;
2. crea una baseline senza LLM;
3. definisci prompt/schema e parametri;
4. esegui dataset normale, limite e ostile;
5. misura valid JSON, source accuracy, factual errors e tempi;
6. analizza tre errori e decidi se il sistema supera il gate;
7. consegna evidence manifest e data-flow.

29.2 Rubrica /100

Dimensione	Punti
Selezione e manifest	15
Baseline e disegno eval	15
Applicazione e gestione errori	20
Metriche e failure analysis	20
Privacy, licenza e prompt injection	15
Spiegazione visuale e difesa	15

Soglia 60. Sicurezza/provenienza almeno 8/15 e nessun dato personale. Bonus entro 5 punti, senza superare 100, per confronto rigoroso con secondo modello.

29.3 Variante AI Engineer

Aggiungere paired bootstrap, retrieval eval, adapter o kernel profilato; riprodurre almeno un'idea ★ della timeline; documentare ablation, costi e regressioni. La sola complessità del codice non assegna punti.